Reinforcement Learning-Based Optimal Setpoint Determination in Energy Utility Systems

Master Thesis

Master of Science in Artificial Intelligence

submitted by

Ashish Thapa

5008270

born on 04 March 1999 in Kaski, Nepal

Department: Wirtschaftsmathematik/Economathematics
Brandenburg University of Technology Cottbus-Senftenberg

Reviewer:	Prof. Dr. rer. nat. habil. Ralf Wunderlich
Reviewer:	Dr.-Ing. Loukas Kyriakidis
Submission date:	19 September 2026

Declaration of independence

I hereby declare that I have written this thesis independently, without unauthorised help and exclusively using the sources indicated. The ideas taken directly or indirectly from external sources (including electronic sources) are labelled as such without exception. The sources used verbatim and in terms of content have been cited in accordance with the recognised rules of scientific work. This work has not yet been submitted for any other review or published elsewhere.

I agree that the thesis will be checked for plagiarism with the help of a plagiarism detection service.

Cottbus, 19 September 2026

Signature of the author

Abstract

Energy utility systems operate under dynamic conditions in which electricity prices, renewable generation, storage states, and interacting process variables affect operating cost, efficiency, reliability, and constraint satisfaction. Improving their operation can also support renewable-energy integration and reduce energy-related carbon emissions. Their setpoints are conventionally determined by solving constrained finite-horizon optimization problems, which depend on accurate system models and can become computationally demanding as system complexity increases. Time-varying inputs, renewable-energy uncertainty, and model–plant mismatch further motivate the investigation of data-driven alternatives. This thesis therefore evaluates whether reinforcement learning can provide effective setpoints with low online computation time for two systems of different complexity: a battery energy storage system and the electrified steam-generation utility system of a cookie production factory.

Both problems are formulated as finite-horizon Markov decision processes, and actor-critic agents are trained using Soft Actor-Critic (SAC) and Twin Delayed Deep Deterministic Policy Gradient (TD3). IPOPT is used to solve the finite-horizon nonlinear programs and serves as the optimization baseline for both systems. Under identical input data, the schedules produced by the RL policies are compared with this baseline in terms of economic objective, feasibility, and online computation time.

For the battery system, TD3 closely approached the economic performance of the optimization baseline while generating schedules approximately 68× faster. For the more complex cookie factory, SAC and TD3 achieved mean absolute relative objective gaps of 6.31% and 5.66%, respectively, while generating setpoints approximately 534× and 645× faster than the optimization baseline. Although SAC had the larger economic gap, it produced smaller F_4-coupling residuals, heat-flow-bound violations, and terminal cyclic deviations, whereas TD3 produced smaller grid-power violations. Both cookie-factory policies produced constraint violations, whereas the optimization baseline remained feasible.

The results show that trained reinforcement-learning policies can generate economically meaningful setpoints with very low online computation time after offline training, although some constraint violations remained in the more complex cookie-factory model.

Zusammenfassung

Beispielzusammenfassung in Deutsch.

Contents

List of Figures vi

List of Tables vii

List of Abbreviations viii

Nomenclature ix

1 Introduction 1

2 Theory and Background 4

2.1 Setpoint Optimization in Energy Utility Systems 4

2.2 Model-Based Optimization 5

2.3 Reinforcement Learning 7

2.3.1 TD3 and SAC 10

3 Battery Energy Storage System 13

3.1 Problem Formulation 13

3.2 Methodology 16

3.2.1 Reinforcement Learning Formulation 16

3.2.2 Training Setup 19

3.2.3 Evaluation Procedure 20

3.3 Numerical Results 22

3.3.1 Comparison with the Optimization Baseline 22

4 Cookie Production Factory 26

4.1 Problem Formulation 26

4.2 Methodology 29

4.2.1 Reinforcement Learning Formulation 30

4.2.2 Training and Evaluation Setup 32

4.3 Numerical Results 33

5 Conclusion and Future Work 36

5.1 Conclusion 36

5.2 Future Work 37

List of Figures

1 Agent–environment interaction using the notation of this thesis. At time t, the agent observes state x_t and selects $a_t \in D_t(x_t)$; the environment evaluates $r_t(x_t, a_t)$ and generates x_{t+1} according to $Q_t(\cdot \mid x_t, a_t)$. 7

2 Schematic of the battery energy storage system. The battery charges from the grid and sells electricity back to the grid; its state of charge SOC_t records the stored energy. 13

3 Visualization of the smooth battery mappings in Eqs. (4) and (5). Panel (a) shows grid-side power as a function of the normalized action for $\mathrm{SOC}_t \in \{0.005, 0.5, 0.995\}$. The black dotted line shows the linear reference $P_t^{\mathrm{S}} = P_{\max} u_t$. Panel (b) shows the conversion from grid-side to storage-side power. The plotted parameters are $P_{\max} = 20\,\mathrm{kW}$, $k = 700$, $\delta = 0.0025$, $\eta = 0.9$, and $\epsilon = 1\,\mathrm{kW}$. 15

4 Illustration of the battery reachability violation and shaped reward as functions of SOC at $t \in \{18, 20, 22\}$. The parameters are $N = 24$, $P_{\max} = 20\,\mathrm{kW}$, $E_{\max} = 200\,\mathrm{kWh}$, $\eta = 0.9$, $\Delta t_{\mathrm{h}} = 1\,\mathrm{h}$, $\mathrm{SOC}_{\mathrm{term}} = 0.5$, and $\lambda_{\mathrm{reach}} = 10$. Panel (a) shows $v_{\mathrm{reach},t}$. Panel (b) uses the idle action $u_t = 0$ at the shown nonterminal steps, for which the baseline reward is $r_t = 0$ and hence $r_t^{\mathrm{reach}} = -\lambda_{\mathrm{reach}} v_{\mathrm{reach},t}$. Hollow circles mark the reachability thresholds. 19

5 Objective values from TD3 and the optimization baseline for ten randomly selected 24-hour comparison windows. The lower panel shows the corresponding gaps between TD3 and the optimization baseline. . 23

6 Illustrative 24-hour battery dispatch comparison between TD3 and the optimization baseline. 24

7 Illustrative 24-hour fixed-threshold battery dispatch. 24

8 Schematic overview of the electrified steam-generation system serving the cookie production factory. Adapted from Fig. 2 of Kyriakidis et al. [9]; the system was originally introduced by Walden et al. [3]. . . . 27

9 Detailed flow chart of the HTHP, TES, SG, and controllable bypasses in the electrified steam-generation system. Adapted from Fig. 3 of Kyriakidis et al. [9] using the notation of this thesis; the system was originally introduced by Walden et al. [3]. 27

10 Objective values from TD3 and the optimization baseline for the first ten matched 24-hour cookie-factory windows. The lower panel shows the corresponding absolute objective gaps between TD3 and the optimization baseline. 34

11 Illustrative 24-hour cookie-factory dispatch comparison between TD3 and the optimization baseline. 35

List of Tables

1 Final SAC and TD3 training configuration used for both the battery and cookie-factory environments. TD3 uses the Stable-Baselines3 default network architecture. 20

2 Comparison between the trained SAC and TD3 policies and the optimization baseline over 1000 random 24-hour windows. Lower (more negative) objective values correspond to higher profit. 22

3 Comparison of the trained SAC and TD3 policies with the optimization baseline over 200 matched 24-hour cookie-factory windows. Lower objective, computation time, and constraint values are better. Constraint quantities are means per window and are reported separately because they have different physical units. 33

List of Abbreviations

Abbreviation	Definition
BESS	Battery Energy Storage System
BO	Bayesian Optimization
CPU	Central Processing Unit
EUR	Euro
HTHP	High-Temperature Heat Pump
HTF	Heat-Transfer Fluid
HTHX	High-Temperature Heat Exchanger
IPOPT	Interior Point OPTimizer
KKT	Karush–Kuhn–Tucker
LTHX	Low-Temperature Heat Exchanger
MDP	Markov Decision Process
MLP	Multilayer Perceptron
NLP	Nonlinear Programming
ReLU	Rectified Linear Unit
RL	Reinforcement Learning
SAC	Soft Actor-Critic
SG	Steam Generator
SOC	State of Charge
TD3	Twin Delayed Deep Deterministic Policy Gradient
TES	Thermal Energy Storage
WT	Wind Turbine

Nomenclature

Symbols are grouped by context. A dash denotes a dimensionless quantity.

Symbol	Definition	Unit
Time, energy, and economic quantities		
t, i, j, ℓ	Time, summation, and component indices; the applicable meaning and range are stated with each equation	–
N	Number of decision steps in the scheduling horizon	–
$\Delta t_{\mathrm{h}}, \Delta t_{\mathrm{s}}$	Duration of one time step in hours and seconds, respectively; $\Delta t_{\mathrm{s}} = 3600\Delta t_{\mathrm{h}}$	h; s
S_t	Spot electricity price	EUR/MWh
$S_t^{\mathrm{buy}}, S_t^{\mathrm{sell}}$	Electricity purchase and sale prices	EUR/MWh
Δ_s	Fixed difference between the purchase and sale prices	EUR/MWh
P_t^{S}	Grid-side electrical power; positive for purchase	kW
P_t^+, P_t^-	Purchased and sold power magnitudes, respectively	kW
W_t	Available wind power	kW
C_t	Grid-electricity cost at one time step	EUR
J_{grid}	Total net grid-electricity cost over the scheduling horizon	EUR
Battery energy storage		
$E_{\max}$	Usable battery energy capacity	kWh
$P_{\max}$	Maximum charging and discharging power	kW
$P_{\mathrm{actual},t}$	Effective storage-side power after efficiency losses	kW
$\widehat{P}_{\mathrm{actual},t}$	Requested storage-side power before limiting it to the available energy	kW
u_t	Normalized battery control command	–
$\mathrm{SOC}_0, \mathrm{SOC}_t$	Initial battery state of charge and state of charge at the beginning of decision step t	–
SOC_N	Terminal state of charge after N decisions	–
$\mathrm{SOC}_{\mathrm{term}}$	Required minimum terminal state of charge	–
$\mathrm{SOC}_t^{\mathrm{reach}}$	Minimum state of charge that preserves terminal-target reachability	–
η	Charging and discharging efficiency	–
k	Steepness parameter of the smooth action and SOC gates	–
δ	Half-width of the normalized battery-action deadband	–
ϵ	Power scale for the smooth efficiency transition	kW
Cookie production factory and thermal storage		
$T_{\mathrm{I},t}, T_{\mathrm{II},t}$	HTHX inlet and outlet temperatures, respectively	°C
$T_{\mathrm{III},t}, T_{\mathrm{IV},t}$	LTHX inlet and outlet temperatures, respectively	°C
$T_{1,t}, T_{4,t}$	TES outlet temperatures during charging and discharging, respectively	°C
$T_{2,t}, T_{3,t}$	Steam-generator inlet and outlet temperatures, respectively	°C
$T_{s,0}, T_{s,t}, T_{s,N}$	Initial, beginning-of-step, and terminal TES temperatures	°C
$\tilde{T}_0$	Prescribed initial thermal-storage temperature	°C
$\dot{m}_{\mathrm{I},t}, \dot{m}_{\mathrm{II},t}$	Per-HTHP HTHX inlet and outlet HTF mass-flow rates; equal in the implemented closed loop	kg/s
m_s	Mass of the thermal-storage medium	kg

Symbol	Definition	Unit
$c_{p,\mathrm{f}}, c_{p,\mathrm{s}}$	Specific heat capacities of the heat-transfer fluid and storage medium	$\mathrm{kJ/(kg\,K)}$
R_t	Compressor-speed-related control variable	–
R_{ref}	Reference value used to select the initial compressor-speed root	–
$\beta_{1,t}, \beta_{2,t}$	TES charging-side and discharging-side bypass settings	–
b_t	Signed cookie-factory TES-mode command	–
δ_β	Half-width of the idle band in the signed TES-mode command	–
$\dot{Q}_{s,\mathrm{ch},t}$	Thermal-storage charging heat rate	kW
$\dot{Q}_{s,\mathrm{dch},t}$	Thermal-storage discharging heat rate	kW
$\dot{Q}_s^{\max}$	Maximum permitted TES charging or discharging heat rate	kW
$P_{\mathrm{S}}^{\max}$	Maximum permitted cookie-factory grid power	kW
$T_s^{\min}, T_s^{\max}$	Lower and upper TES temperature limits	°C
ΔT_s	TES temperature span, $T_s^{\max} - T_s^{\min}$	°C
$T_{\mathrm{II,req},t}$	HTHX outlet temperature required to satisfy the F_4 coupling relation	°C
$\epsilon_{\mathrm{ch}}, \epsilon_{\mathrm{dch}}$	Thermal-storage charging and discharging effectiveness	–
γ	Upper bound on the product of charging and discharging heat rates	kW^2
F_{HTHP}	Surrogate model for electrical power demand of one HTHP unit	kW
$F_{\mathrm{HTHX}}, F_{\mathrm{LTHX}}$	Surrogate models for high- and low-temperature heat-exchanger outlet temperatures	°C
$\kappa_{j,t}$	Coefficient of R^j in the HTHX outlet-temperature surrogate with the other inputs fixed at time t; $j = 0, \ldots, 3$	°C
F_4, F_5	Labels for the charging-side coupling and discharging-side mixing relations in Eqs. (9) and (10)	n/a
Model-based optimization		
$\xi, \xi^\star$	Decision vector and optimal decision vector	*
Ω	Domain of the decision variables	n/a
n_ξ, n_h, n_c	Numbers of decision variables, equality constraints, and inequality constraints	–
$f(\xi)$	Objective function	*
$h(\xi), c(\xi)$	Aggregate equality- and inequality-constraint vectors	*
$c_j(\xi)$	jth inequality-constraint function	*
s, s_j	Slack vector and its jth component	*
μ_{b}	Logarithmic barrier parameter	*
y, z	Equality- and inequality-constraint multipliers	*
J_h, J_c	Jacobian matrices of the constraint functions	*
$\mathsf{S} = \mathrm{diag}(s)$	Diagonal matrix of slack variables	*
Reinforcement learning and evaluation		
E, A	State and action spaces of the finite-horizon MDP	n/a
$D_t(x)$	Admissible actions in state x at time step t	n/a
$Q_t(\cdot \mid x, a)$	Transition probability kernel at time step t	–
$p_t(x' \mid x, a)$	Transition probability density, when it exists	*
$g_N(x)$	Separate terminal-reward function	**
x, x_t, x'	Generic state, state at time step t, and next state	*
X_t, A_t	Random state and action at time step t	*

Symbol	Definition	Unit
$a, a_t, a_t^{\text{phys}}$	Generic action, policy action at time t, and corresponding physical setpoints	*
a'_{t+1}	Next action sampled from the SAC policy for the critic target	*
$\mathcal{M}$	Map from policy actions to physical setpoints	n/a
$\pi, \pi_t(\cdot \mid x), \pi_\theta(a \mid x)$	Policy, time-t conditional action distribution, and parameterized stochastic policy	*
π_{b}	Behavior policy used to collect transitions	n/a
$\mu_\theta(x), \mu_{\bar{\theta}}(x)$	Deterministic TD3 actor and target actor	*
θ	Actor-parameter vector for TD3 or SAC	*
ϕ	Generic critic-parameter vector	*
ϕ_1, ϕ_2	TD3 critic-parameter vectors	*
ψ_1, ψ_2	SAC critic-parameter vectors	*
$\bar{\theta}, \bar{\phi}_1, \bar{\phi}_2$	TD3 target-network parameter vectors	*
$\bar{\psi}_1, \bar{\psi}_2$	SAC target-critic parameter vectors	*
$\mathcal{D}$	Replay buffer; distinct from the admissible-action set $D_t(x)$	n/a
d_t	Episode-end indicator	–
$r_t(x, a), r_t(x_t, a_t), r_t$	Stage-reward function, realized value, and shorthand for the realized value	**
$r_t^{\text{cookie}}(x_t, a_t)$	Cookie-factory stage reward	**
$r_t^{\text{reach}}(x_t, a_t)$	Battery reward including the terminal-SOC reachability penalty	**
G_t	Finite-horizon return from time step t	**
γ_{RL}	Reward discount factor; set to 1 in all experiments	–
v_t^π, q_t^π	State-value and action-value functions under policy π	**
$v_t^\star, q_t^\star$	Optimal state-value and action-value functions	**
$\mathcal{A}_t^\pi(x, a)$	Advantage function, $q_t^\pi(x, a) - v_t^\pi(x)$	**
q_ϕ	Generic parameterized action-value critic	**
q_{ψ_1}, q_{ψ_2}	Two parameterized SAC critics	**
$q_{\bar{\psi}_1}, q_{\bar{\psi}_2}$	SAC target critics	**
q_{ϕ_1}, q_{ϕ_2}	Two parameterized TD3 critics	**
$q_{\bar{\phi}_1}, q_{\bar{\phi}_2}$	TD3 target critics	**
$\widehat{v}^{\text{tar}}$	Continuation-value estimate used in a TD target	**
y_t	Generic one-step TD learning target	**
$y_t^{\text{TD3}}, y_t^{\text{SAC}}$	TD3 and SAC critic learning targets	**
δ_t^{TD}	Temporal-difference error	**
ζ	TD3 target-policy smoothing noise	*
$L(\phi_j), L(\psi_j)$	Mean squared TD-error losses of TD3 and SAC critics	(reward units)^2
$J_{\text{PG}}, J_{\text{TD3}}, J_{\text{SAC}}$	Generic policy-gradient, TD3 actor, and SAC policy objectives	**
α	Weight of the SAC entropy term	**
$\mathcal{H}$	Policy entropy	–
τ	Target-network soft-update coefficient	–
$[\cdot]_+$	Positive-part operator	same as its argument
$v_{\ell,t}$	Constraint violation in category ℓ at time step t	*
$v_{\text{T},t}$	Terminal violation, including the final-step indicator; zero before the episode ends	– (battery); °C (cookie factory)
$v_{\text{reach},t}$	Battery terminal-SOC reachability violation	–
$v_{s,t}, v_{\text{I},t}, v_{F_4,t}$	Cookie-factory TES-temperature, inlet-temperature, and coupling violations	°C

Symbol	Definition	Unit
$v_{Q,t}, v_{P,t}$	Cookie-factory heat-flow and grid-power violations	kW
$\lambda_\ell, \lambda_{\mathrm{T}}$	Constraint and terminal penalty weights	reward units per violation unit
$\lambda_s, \lambda_{\mathrm{I}}, \lambda_Q$	Cookie-factory TES-temperature, inlet-temperature, and heat-flow penalty coefficients in Eq. (15)	**
$\lambda_P, \lambda_{F_4}, \lambda_{\mathrm{cyc}}$	Cookie-factory grid-power, coupling, and cyclic-terminal penalty coefficients in Eq. (15)	**
$\lambda_{\mathrm{SOC}}, \lambda_{\mathrm{reach}}$	Terminal-SOC and reachability penalty weights	**
R_{ep}	Cumulative undiscounted episode reward	**
$V_{\mathrm{tot},\ell}$	Sum of recorded violation magnitudes for constraint category ℓ	*
f_{viol}	Fraction of time steps with constraint violations	–
$\varepsilon_{\mathrm{viol},\ell}$	Numerical violation-detection tolerance for constraint category ℓ	*
$\mathbb{E}, \mathbb{E}_\pi, \mathbb{E}_{\mathcal{D}}$	Expectation, expectation under policy π, and expectation over replay-buffer samples	n/a
$\mathbf{1}$	Vector of ones	–
$\mathbb{1}, \mathbb{1}_{\{\cdot\}}$	Indicator function and event-indicator notation	–

* varies
** reward units

1 Introduction

Modern energy utility systems supply the electricity and heat needed to sustain industrial production and other essential activities. Their efficient operation is important for meeting energy demand while limiting operating costs and environmental impacts. Key objectives of industrial energy management include reducing operating costs and CO_2 emissions, improving energy efficiency, and integrating renewable energy sources. Achieving these objectives requires coordinated decisions about energy conversion, storage, and supply while satisfying the requirements of the processes being served.

These operating decisions are shaped by dynamic conditions and multiple sources of uncertainty. Electricity prices vary over time, renewable generation is weather dependent, and storage systems couple current decisions with future operating flexibility [1]. At the same time, industrial energy systems often exhibit nonlinear component behavior and include operating and coupling constraints that must be respected at every time step. The resulting setpoint selection problem is therefore not only an economic problem, but also a constrained sequential decision-making problem.

In many engineering applications, such setpoints are determined by solving constrained finite-horizon optimization problems. This approach is attractive because physical constraints and objective functions can be formulated explicitly, and mature solvers can compute high-quality solutions for well-defined models. For energy systems, nonlinear programming methods such as IPOPT [2] are commonly used when component models, storage dynamics, and operational constraints are nonlinear [3]. However, deterministic global optimization can become computationally demanding for large-scale or nonconvex problems, making it difficult to obtain a globally optimal solution within a practical time limit [4]. Stochastic global optimization methods can also require many objective-function evaluations and exhibit slow convergence, with computational cost increasing as the problem dimension grows [5]. In industrial energy management, forecast uncertainty, optimization-horizon length, and computation time per rolling-horizon iteration are additional factors that influence system performance [6].

Reinforcement learning (RL) provides a different perspective. Unlike classical control and optimization methods that solve a trajectory-optimization problem from scratch at each operating horizon, RL agents learn a policy or action-value function gradually through interactive experience. Once trained, this learned representation enables rapid selection of actions from the current observation [7]. This makes RL attractive for real-time or repeated decision-making problems, especially when the system is complex, nonlinear, and operates in a changing environment [8]. Nevertheless, RL does not automatically guarantee feasibility or optimality. For engineering use, a learned policy must be evaluated not only by its reward, but also by its operating cost, constraint satisfaction, and computation time relative to established optimization methods.

This thesis investigates whether RL can be used for optimal setpoint determination in energy utility systems of different complexity. Two case studies are considered to examine how the number of control variables, nonlinear component behavior, and

coupling between operational constraints affect the applicability of RL. The first is a battery energy storage system (BESS), which represents a comparatively simple storage-control problem with a single charging or discharging command, state-of-charge dynamics, and time-varying electricity prices. The second is the electrified steam-generation utility system of a cookie production factory, incorporating wind power and thermal energy storage. Based on the benchmark systems described by Walden et al. and Kyriakidis et al. [3, 9], this case has greater complexity because it combines multiple continuous control variables, nonlinear heat-pump and heat-transfer behavior, and coupled mass-flow, temperature, and storage constraints. Comparing the two cases allows the benefits and limitations of RL to be assessed across a simpler storage system and a more complex industrial energy utility system.

The central research question of this thesis is:

> *Can reinforcement learning determine feasible and cost-effective operating setpoints for energy utility systems of varying complexity?*

This question is addressed through the following subquestions:

- How can the setpoint optimization problem be formulated as a Markov decision process with suitable states, actions, and rewards?
- Can deep reinforcement-learning policies learn continuous setpoint decisions that respect the system dynamics and operational constraints of the two systems?
- How does the learned policy compare with the optimization baseline in terms of operating cost, constraint satisfaction, and computation time?
- Under which conditions does RL offer a practical advantage, and where do classical optimization methods remain superior?

For each case study, the setpoint optimization problem is first formulated as a Markov decision process (MDP). Deep neural networks are used to parameterize the RL policies that map system observations to continuous setpoint decisions. The resulting policies are evaluated through deterministic rollouts in reinforcement-learning environments, and the resulting schedules are compared with the optimization baseline. The comparison is based on total grid-electricity cost, constraint violations, and computation time. This separation is important because a policy with high cumulative reward is only useful if it remains feasible and economically meaningful under the original engineering objective.

The remainder of this thesis is structured as follows: Section 2 establishes the theoretical foundations of setpoint optimization and reinforcement learning, introduces IPOPT as the optimization baseline, and describes the TD3 and SAC algorithms considered in this work. Section 3 presents the battery energy storage system, followed by its problem formulation, methodology, and numerical results. The implementation, training procedure, and evaluation metrics shared by both case studies are introduced within the battery methodology. Section 4 then presents the cookie production factory in the same order: problem formulation, methodology,

and numerical results. Each case study compares the learned policies with the optimization baseline in terms of economic performance, constraint satisfaction, and computation time. Finally, Section 5 summarizes the main findings, identifies the limitations of the study, and outlines directions for future work.

2 Theory and Background

This section provides the theoretical basis for the setpoint optimization problem studied in this thesis. The central question is how operating setpoints in energy utility systems can be selected over time so that costs are reduced while physical and operational limits are respected. The same problem is studied through two computational approaches: direct solution of a finite-horizon constrained nonlinear program for each input window and action selection by RL policies trained offline. The section first introduces the model-based optimization formulation and its solution with IPOPT. It then presents the fundamental concepts of RL and the MDP formulation [7], followed by descriptions of the TD3 [10] and SAC [11] algorithms for continuous control evaluated in this work.

Here, RL is not treated as a substitute for physical constraints. Instead, the setpoint optimization problem is formulated as an MDP, with feasibility enforced where possible through action bounds, state-dependent mappings, and system dynamics, and otherwise discouraged through reward penalties. This distinction is important when comparing RL with the optimization baseline, because a low-cost policy is only useful if its decisions remain feasible.

2.1 Setpoint Optimization in Energy Utility Systems

Energy utility systems are operated by choosing setpoints that determine how components such as storage units, heat pumps, valves, and bypasses are controlled. A setpoint describes the desired value of a control variable. In the systems considered here, this may be the charging or discharging power of a battery, the bypass variable $\beta_{1,t}$, or a mass flow rate. These choices are not just technical details; they directly affect operating cost, energy efficiency, constraint satisfaction, and overall system reliability and lifetime [12].

Setpoint determination is also inherently sequential. The best operating point at one time step cannot be chosen independently of what happened before or what may be needed later. Storage systems make this especially clear. Charging a battery or a thermal energy storage system can increase future flexibility, but it may also increase the current electricity cost. Discharging can reduce current grid consumption or generate sale revenue, but it leaves less stored energy for later. Good setpoint decisions must therefore account for inter-temporal dependencies and physical operating limits [3]. Each action affects both the immediate economic outcome and the feasible decisions available at later time steps; consequently, its quality must be evaluated over the full planning horizon rather than solely at the current time step [13].

In practical operation, the controller also has to react to external signals such as electricity prices, renewable generation, production demand, and ambient conditions. Since these signals change over time, fixed rule-based strategies can become inefficient, and the operation can be formulated as a finite-horizon decision problem with time-varying boundary conditions [8, 14]. This motivates two solution approaches for the same setpoint determination task: solving a constrained finite-horizon problem directly and applying a policy learned through reinforcement

learning.

2.2 Model-Based Optimization

Model-based optimization provides a natural way to describe setpoint selection. In general, optimization means selecting the best decision from a set of feasible alternatives according to a defined objective. For energy systems, the objective is often to minimize operating costs, energy consumption, or CO_2 emissions, while the feasible set is determined by physical equations and operational limits [3, 6]. A constrained optimization problem can be written as [15]:

$$\min_{\xi \in \Omega} f(\xi) \quad \text{s.t.} \quad h(\xi) = 0,\ c(\xi) \leq 0, \qquad \text{..... (1)}$$

Here, $\xi \in \Omega \subseteq \mathbb{R}^{n_\xi}$ denotes the decision vector, $f : \mathbb{R}^{n_\xi} \to \mathbb{R}$ is the objective function, $h : \mathbb{R}^{n_\xi} \to \mathbb{R}^{n_h}$ represents the equality constraints, and $c : \mathbb{R}^{n_\xi} \to \mathbb{R}^{n_c}$ represents the inequality constraints. The solution $\xi^\star$ is optimal if it minimizes the objective while satisfying all constraints.

For finite-horizon setpoint optimization, the decision vector contains the control actions for several time steps. The objective is usually the accumulated operating cost over the horizon. The constraints describe system dynamics, energy balances, storage bounds, and terminal requirements. This time-coupled structure is important because a state such as battery state of charge or thermal storage temperature depends on previous decisions.

The difficulty of an optimization problem depends strongly on its mathematical structure. Linear and mixed-integer linear formulations are widely used in energy-system applications because simplified models can often be solved reliably. However, realistic energy conversion systems frequently include nonlinear component behavior, continuous control variables, and coupled dynamic constraints. Such systems lead to nonlinear programming (NLP) formulations, which are generally harder to solve and may contain local optima [15]. As the number of time steps, decision variables, nonlinear component models, and coupling constraints increases, the optimization problem generally becomes larger and more difficult to solve within a short operational time window [9].

Another important distinction is whether the problem is deterministic or stochastic. In deterministic optimization, all relevant parameters, such as electricity prices, wind availability, and demand, are assumed to be known over the optimization horizon. Stochastic optimization represents uncertain quantities through random variables or scenarios. This is closer to real operation, but it also increases model size and computational complexity.

Nonlinear constrained problems are usually solved with dedicated numerical solvers. Interior-point methods are common in this setting. Wächter and Biegler introduce IPOPT as an open-source solver for large-scale nonlinear programming that uses a primal-dual interior-point method with a filter line-search strategy [2].

More advanced hybrid optimization methods, such as Bayesian Optimization-Interior Point OPTimizer (BO-IPOPT), have been developed for complex nonlinear, nonconvex, and constrained optimization problems. BO-IPOPT combines the global

exploration capability of Bayesian optimization with the efficient local convergence and constraint handling of IPOPT. The method has been evaluated for the optimal operation of an industrial energy conversion system and subsequently enhanced for constrained problems of varying dimensionality, multimodality, and nonlinearity [9, 16]. In this thesis, IPOPT is used to solve the finite-horizon nonlinear programs and serves as the optimization baseline. The evaluation of BO-IPOPT as an additional global optimization method is left for future work.

IPOPT handles the inequality constraints in Eq. (1) by introducing the slack vector $s = (s_1, \ldots, s_{n_c})^\top$. Each inequality $c_j(\xi) \leq 0$ is then expressed as $c_j(\xi)+s_j = 0$ with $s_j \geq 0$. To keep the slack variables strictly positive during the optimization, the interior-point formulation augments the objective with a logarithmic barrier parameterized by $\mu_\mathrm{b} > 0$. This yields the barrier subproblem [2]

$$\begin{aligned} \min_{\xi,s} \quad & f(\xi) - \mu_\mathrm{b} \sum_{j=1}^{n_c} \ln(s_j) \\ \text{s.t.} \quad & h(\xi) = 0, \qquad c(\xi) + s = 0, \qquad s > 0. \end{aligned}$$

The logarithmic barrier prevents the iterates from reaching the constraint boundary. IPOPT solves a sequence of such subproblems using a primal-dual Newton method. With equality multipliers y and inequality multipliers z, the perturbed Karush-Kuhn-Tucker(KKT) conditions are

$$\begin{aligned} \nabla f(\xi) + J_h(\xi)^\top y + J_c(\xi)^\top z = 0, \\ h(\xi) = 0, \qquad c(\xi) + s = 0, \\ \mathsf{S}z = \mu_\mathrm{b}\mathbf{1}, \qquad s > 0,\ z > 0, \end{aligned}$$

where J_h and J_c are the Jacobians of the equality and inequality constraints, respectively, and $\mathsf{S} = \mathrm{diag}(s)$. As μ_b is reduced towards zero, the perturbed complementarity condition $\mathsf{S}z = \mu_\mathrm{b}\mathbf{1}$ approaches the KKT complementarity condition of the original constrained problem [2].

Nocedal and Wright note that the performance of nonlinear optimization algorithms depends on formulation quality, particularly the constraint structure, variable scaling, and initial point [15].

The KKT conditions are not specific to IPOPT. They are also relevant when an RL-generated schedule is evaluated as a solution to the original constrained problem. To be described as locally optimal, the RL schedule would need to satisfy the original constraints and the KKT conditions.

The RL methods used in this thesis do not explicitly enforce the KKT conditions. Therefore, the RL schedules are not claimed to be optimal. Their objective values and constraint violations are reported separately and compared with the IPOPT optimization baseline. The main motivation for investigating RL is its potential to generate control decisions quickly once training has been completed offline.

The following section introduces the fundamental principles of RL and its formulation for setpoint optimization.

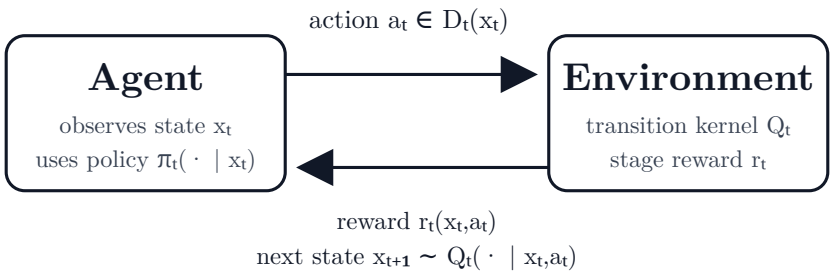


Figure 1: Agent–environment interaction using the notation of this thesis. At time t, the agent observes state x_t and selects $a_t \in D_t(x_t)$; the environment evaluates $r_t(x_t, a_t)$ and generates x_{t+1} according to $Q_t(\cdot \mid x_t, a_t)$.

2.3 Reinforcement Learning

Reinforcement learning (RL) offers a different view of the same setpoint problem. Instead of solving an optimization problem from scratch at every decision time, an agent learns from repeated interaction with an environment. At each time step, the agent observes the current state, selects an action, receives a reward, and observes the next state. Over time, it adapts its policy so that actions leading to high long-term reward become more likely [7].

Following Bäuerle and Rieder [17], a non-stationary Markov decision model with planning horizon $N \in \mathbb{N}$ consists of the data

$$(E, A, D_t, Q_t, r_t, g_N)\,,$$

with the following meaning for $t = 0, 1, \ldots, N-1$:

- E is the state space and A is the action space,
- $D_t(x) \subseteq A$ is the set of admissible actions in state x at time t,
- $Q_t(\cdot \mid x, a)$ is the transition kernel,
- $r_t(x, a)$ is the stage-reward function, and
- $g_N(x)$ is the terminal-reward function.

Under the Markov property, the current state contains all information required to predict the next state, given the selected action. A non-stationary policy is a sequence $\pi = (\pi_t)_{t=0}^{N-1}$, where $\pi_t(\cdot \mid x)$ is the conditional action distribution in state x at time t. Thus, π_t defines the action-selection rule at time t, conditional on the current state x. For a deterministic policy, this distribution assigns all probability to one action. Following the finite-horizon convention of Bäuerle and Rieder [17], the total return from time t is

$$G_t = \sum_{i=t}^{N-1} \gamma_{\mathrm{RL}}^{i-t} r_i(X_i, A_i) + \gamma_{\mathrm{RL}}^{N-t} g_N(X_N),$$

$$\gamma_{\mathrm{RL}} = 1 \quad \text{for the finite-horizon problems considered here.}$$

Because $\gamma_{\mathrm{RL}} = 1$ in all experiments, all discount powers equal unity, and the ex-

pression reduces to the undiscounted total-reward form used by Bäuerle and Rieder. The terminal reward $g_N(X_N)$ is evaluated at the terminal state. In the implemented environments, $g_N \equiv 0$ because terminal penalties are included in the stage reward returned with the final transition.

Bellman equations and RL solution approaches. The learning objective is to find a policy that maximizes the expected return. The value function and action-value function of a policy π are defined as

$$v_t^\pi(x) = \mathbb{E}_\pi[G_t \mid X_t = x]\,, \qquad q_t^\pi(x,a) = \mathbb{E}_\pi[G_t \mid X_t = x, A_t = a]\,.$$

The value function evaluates a state before an action is selected, whereas the action-value function evaluates a particular state–action pair. For a fixed policy, they satisfy the Bellman expectation equations

$$q_t^\pi(x,a) = r_t(x,a) + \gamma_{\mathrm{RL}} \int_E v_{t+1}^\pi(x')\, Q_t(\mathrm{d}x' \mid x,a), \qquad \text{..... (2)}$$

$$v_t^\pi(x) = \int_{D_t(x)} q_t^\pi(x,a)\, \pi_t(\mathrm{d}a \mid x), \qquad t = 0,\dots,N-1, \qquad \text{..... (3)}$$

Here, x' is a dummy variable representing a possible next state. If densities exist, these measures may be written as $Q_t(\mathrm{d}x' \mid x,a) = p_t(x' \mid x,a)\,\mathrm{d}x'$ and $\pi_t(\mathrm{d}a \mid x) = \pi_t(a \mid x)\,\mathrm{d}a$; for discrete state or action spaces, the corresponding integrals become sums.

With terminal condition $v_N^\pi(x) = g_N(x)$, Eq. (2) separates the value of an action into its immediate reward and its expected discounted future value. Equation (3) averages the action values according to the policy. For a deterministic policy, this average reduces to the value of the single selected action.

The Bellman optimality equations describe the best attainable value rather than the value under a fixed policy:

$$\begin{aligned} q_t^\star(x,a) &= r_t(x,a) + \gamma_{\mathrm{RL}} \int_E v_{t+1}^\star(x')\, Q_t(\mathrm{d}x' \mid x,a), \\ v_t^\star(x) &= \sup_{a \in D_t(x)} q_t^\star(x,a), \end{aligned} \qquad t = 0,\dots,N-1,$$

with terminal condition $v_N^\star(x) = g_N(x)$. Thus, the Bellman expectation equations evaluate a specified policy, while the Bellman optimality equations select the action with the greatest value. Both express the same recursive relationship: current value equals immediate reward plus the value of subsequent decisions [7, 17].

Reinforcement learning and temporal-difference learning. Model-free RL learns a decision rule from sampled transitions without requiring the transition kernel Q_t in explicit form. A transition $(x_t, a_t, r_t, x_{t+1}, d_t)$ contains the current state and action, the resulting reward and successor state, and a terminal indicator d_t. Here and in the learning updates below, r_t is shorthand for the realized stage reward $r_t(x_t, a_t)$. Although the samples in this thesis are generated by simulation, the learning algorithms update their neural networks from these transition tuples rather than

from the simulator equations. The implemented state includes the position within the scheduling horizon, so the neural-network policies and critics do not require a separate time subscript.

Temporal-difference (TD) learning updates a value approximation using the observed reward and an estimate of the value at the next state. For an action-value approximation q_ϕ with parameter vector ϕ, a generic one-step TD target is

$$y_t = r_t + \gamma_{\mathrm{RL}}(1 - d_t)\widehat{v}^{\mathrm{tar}}(x_{t+1}),$$

and the corresponding TD error is

$$\delta_t^{\mathrm{TD}} = y_t - q_\phi(x_t, a_t).$$

The target combines the observed immediate reward with an estimate of the remaining return. The notation $\widehat{v}^{\mathrm{tar}}$ denotes this estimate at x_{t+1}. The factor $(1 - d_t)$ removes the continuation value at a terminal transition. The superscript tar indicates that the continuation estimate is treated as fixed during the gradient step and is typically constructed using slowly updated target networks. The action-value approximation is trained by minimizing the mean squared TD error. This update builds directly on the Bellman relationship while learning from sampled transitions [7].

Policy gradients and actor–critic learning. A critic estimates the long-term quality of actions, but a continuous control problem also requires an efficient way to select an action. Directly maximizing $q_\phi(x, a)$ over every possible continuous action at every decision step can be costly. Policy-gradient methods instead represent the policy by an actor with parameters θ and adjust those parameters in the direction of higher expected return.

For a stochastic policy $\pi_\theta(a \mid x)$, the policy-gradient principle weights the score $\nabla_\theta \log \pi_\theta(a \mid x)$ by an estimate of how advantageous the sampled action was:

$$\nabla_\theta J_{\mathrm{PG}}(\theta) \approx \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid x)\, \mathcal{A}_t^\pi(x, a)]\,, \qquad \mathcal{A}_t^\pi(x, a) = q_t^\pi(x, a) - v_t^\pi(x).$$

Here, $J_{\mathrm{PG}}(\theta)$ is the generic policy-gradient objective and $\mathcal{A}_t^\pi$ is the advantage function; A remains the action space. The baseline $v_t^\pi(x)$ does not change the expected gradient but can reduce its variance. For a deterministic actor $a = \mu_\theta(x)$, the critic supplies the action derivative and the chain rule gives

$$\nabla_\theta J_{\mathrm{PG}}(\theta) \approx \mathbb{E}\Big[\nabla_a q_\phi(x, a)|_{a=\mu_\theta(x)}\, \nabla_\theta \mu_\theta(x)\Big]\,.$$

Actor–critic methods combine these two roles: the critic is updated from TD targets, and the actor is updated using the critic as its learning signal. The actor may represent either a deterministic action-selection rule or a stochastic policy. This structure supports learning policies for the continuous battery-power, mass-flow, and storage-mode setpoints considered in this thesis [7].

On-policy and off-policy learning. The on-policy/off-policy distinction concerns the relationship between the policy that generated the data and the policy

being improved. Let π_{b} denote the behavior policy used to collect a transition and π_θ the current target policy being optimized. In on-policy learning, the updates use data generated by the current policy, so $\pi_{\mathrm{b}} = \pi_\theta$ apart from any exploration mechanism regarded as part of that policy. As the policy changes, older samples become less representative of the current policy.

In off-policy learning, the behavior and target policies may differ. This permits transitions generated by older policies, or by policies with additional exploration noise, to be stored in a replay buffer $\mathcal{D}$ and reused in many minibatch updates. The sampled action a_t in a replay transition was produced by the behavior policy at data-collection time, while the action used in the bootstrapped target is generated from the current actor or a slowly updated target actor, depending on the algorithm. Reusing decorrelated experience improves data efficiency, although the distribution mismatch can make learning less stable and motivates devices such as target networks and conservative critic targets.

2.3.1 TD3 and SAC

Twin Delayed Deep Deterministic Policy Gradient (TD3): TD3 represents its deterministic policy by an actor $\mu_\theta : E \to A$, where θ denotes the actor parameters. The output $\mu_\theta(x)$ is the action selected in state x; it is not an action probability. In terms of the general policy notation $\pi_t(\cdot \mid x)$ introduced above, the deterministic TD3 policy assigns all probability to the single action $a = \mu_\theta(x)$. The state contains the position within the scheduling horizon, so the same actor network can be used at every time step without an explicit time subscript.

TD3 learns two separate action-value approximations, q_{ϕ_1} and q_{ϕ_2}, with critic-parameter vectors ϕ_1 and ϕ_2. For a transition $(x_t, a_t, r_t(x_t, a_t), x_{t+1}, d_t)$ sampled from the replay buffer $\mathcal{D}$, their common learning target is

$$y_t^{\mathrm{TD3}} = r_t + \gamma_{\mathrm{RL}}(1 - d_t) \min_{j \in \{1,2\}} q_{\bar{\phi}_j}(x_{t+1}, \mu_{\bar{\theta}}(x_{t+1}) + \zeta)\,,$$

where $d_t = 1$ for a terminal transition, bars denote target-network parameters, and ζ is clipped target-policy smoothing noise. Taking the smaller of the two target estimates reduces the propagation of overestimated action values. Both critics are fitted to this target according to

$$L(\phi_j) = \mathbb{E}_{\mathcal{D}}\Big[\big(q_{\phi_j}(x_t, a_t) - y_t^{\mathrm{TD3}}\big)^2\Big]\,, \qquad j \in \{1, 2\}.$$

TD3 further stabilizes learning by updating the actor and target networks less frequently than the critics [10].

$$J_{\mathrm{TD3}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}}\left[q_{\phi_1}\big(x, \mu_\theta(x)\big)\right].$$

Here, $x \sim \mathcal{D}$ is shorthand for sampling the state component x from a transition stored in the replay buffer. The calligraphic replay buffer $\mathcal{D}$ is distinct from the admissible-action set $D_t(x)$. The actor is trained to maximize J_{TD3} using the first critic q_{ϕ_1}. The second critic is not redundant: it is used together with the first critic in the minimum that defines the critic target above. The subscript 1 merely identifies which of the two critics supplies the actor gradient.

Soft Actor-Critic (SAC): SAC instead uses a stochastic actor, $\pi_\theta(a \mid x)$, that defines a distribution over continuous actions. As with TD3, the position in the scheduling horizon is included in x, so a separate time index is not required on the neural-network policy. In addition to maximizing expected return, SAC includes an entropy term that encourages exploration while training. Like TD3, SAC uses two critics and experience replay; however, its stochastic policy and entropy objective distinguish it from the deterministic TD3 policy [11]. For a replay transition, a next action $a'_{t+1} \sim \pi_\theta(\cdot \mid x_{t+1})$ is sampled from the current policy. The entropy-adjusted critic target is

$$y_t^{\mathrm{SAC}} = r_t + \gamma_{\mathrm{RL}}(1 - d_t)\left[\min_{j\in\{1,2\}} q_{\bar{\psi}_j}(x_{t+1}, a'_{t+1}) - \alpha \log \pi_\theta(a'_{t+1} \mid x_{t+1})\right].$$

The target critics $q_{\bar{\psi}_1}$ and $q_{\bar{\psi}_2}$ are slowly updated copies of the learned critics. Each critic minimizes

$$L(\psi_j) = \mathbb{E}_{\mathcal{D}}\left[\left(q_{\psi_j}(x_t, a_t) - y_t^{\mathrm{SAC}}\right)^2\right], \qquad j \in \{1, 2\}.$$

The entropy term implements a soft TD target that values both predicted return and policy randomness. At the trajectory level, SAC maximizes an entropy-regularized expected return. In the implemented off-policy algorithm, the actor update is estimated using states from the replay buffer and actions sampled from the current policy:

$$J_{\mathrm{SAC}}(\theta) = \mathbb{E}_{\substack{x\sim\mathcal{D} \\ a\sim\pi_\theta(\cdot|x)}}\left[\min_{j\in\{1,2\}} q_{\psi_j}(x, a) - \alpha \log \pi_\theta(a \mid x)\right].$$

Here, $x \sim \mathcal{D}$ denotes sampling the state component from a replay-buffer transition, whereas $a \sim \pi_\theta(\cdot \mid x)$ denotes sampling an action from the current policy for that state. The functions q_{ψ_1} and q_{ψ_2} are the two SAC critics, and their minimum reduces positive estimation bias. The coefficient α weights the entropy contribution. The actor is trained to maximize J_{SAC}; equivalently, implementations written as minimization use the negative of this expression.

For a state x, the policy entropy is defined as

$$\mathcal{H}\big(\pi_\theta(\cdot \mid x)\big) = -\mathbb{E}_{a\sim\pi_\theta(\cdot|x)}\left[\log \pi_\theta(a \mid x)\right].$$

Thus, entropy is an expectation over the policy's action distribution rather than a quantity associated with a single action. During SAC training, an action $a \sim \pi_\theta(\cdot \mid x)$ is sampled from the current policy. The quantity $-\log \pi_\theta(a \mid x)$ is a stochastic single-sample estimate of the entropy term: although one sample is generally not equal to the exact entropy, its expectation over actions drawn from the policy equals $\mathcal{H}(\pi_\theta(\cdot \mid x))$. Such stochastic estimates allow SAC to optimize the entropy-regularized objective using minibatch gradient updates [11].

Both TD3 and SAC are model-free, off-policy actor–critic algorithms. TD3 collects data using its deterministic actor plus exploration noise, but trains from replayed transitions and forms its next action with a target actor. SAC likewise trains its critics and actor from replay-buffer states while sampling target or actor-update actions from its current stochastic policy. Consequently, calling the methods off-policy does not mean that their actor updates ignore the current policy; it means that the stored states and actions used for learning need not have been generated by

the policy currently being optimized. Their replay buffers provide the sampled transitions, their critics learn through TD errors, and their actors produce continuous setpoint decisions [10, 11].

The two algorithms therefore provide a comparison between deterministic and stochastic actor–critic policies for continuous setpoint optimization. Their performance is evaluated using operating cost, constraint satisfaction, and computation time.

3 Battery Energy Storage System

This case study considers a battery energy storage system as the simpler benchmark, with one control command and state-of-charge dynamics. The system model and economic objective are presented first, followed by the reinforcement-learning methodology and numerical results.

3.1 Problem Formulation

A battery energy storage system (BESS) is an electrochemical device that stores electrical energy for later use and provides grid services such as energy arbitrage, peak shaving, and short-term balancing. A typical BESS is characterized by its usable energy capacity, maximum charge/discharge power, round-trip efficiency, and state-of-charge (SOC) operating limits. Real-world operation must balance economic objectives (e.g., buying electricity at low prices and selling it at high prices) with engineering constraints (SOC bounds, power limits, efficiency losses).

BESS operation presents several challenges. Decisions are inherently time-coupled because current charging and discharging changes future availability. In addition, uncertain exogenous signals (market prices) require policies that are robust to forecast errors, and hard feasibility constraints must be enforced at every time step. These properties make BESS an illustrative case study for comparing optimization-based and reinforcement-learning approaches to real-time setpoint determination in energy systems.

A battery energy storage system provides flexible power by buying electricity when prices are low and selling it back to the grid when prices are high. This price-arbitrage operation maximizes profit. A fixed buy–sell price spread $\Delta_s \geq 0$ can be represented by defining the purchase price as $S_t^{\text{buy}} = S_t$ and the selling price as $S_t^{\text{sell}} = S_t - \Delta_s$. In the numerical experiments considered here, $\Delta_s = 0$, so the same time-varying electricity price is used for purchases and sales.

The SOC dynamics are modeled as

$$\text{SOC}_{t+1} = \text{SOC}_t + \frac{P_{\text{actual},t}\,\Delta t_{\text{h}}}{E_{\text{max}}},$$

Here, SOC_t is the state of charge at the beginning of decision step t, SOC_0 is the

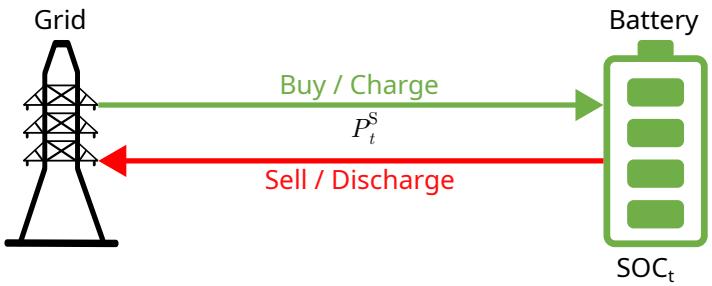


Figure 2: Schematic of the battery energy storage system. The battery charges from the grid and sells electricity back to the grid; its state of charge SOC_t records the stored energy.

initial state, and SOC_N is the terminal state after N decisions. Furthermore, $P_{\mathrm{actual},t}$ is the effective storage-side power after efficiency losses, $E_{\max}$ is the usable energy capacity, and Δt_{h} is the time-step duration in hours. The sign convention is $P_t^{\mathrm{S}} > 0$ for charging and grid purchase, and $P_t^{\mathrm{S}} < 0$ for discharging and electricity sale. The superscript S identifies power exchanged with the electricity supply grid. The mathematical formulation uses the following smooth power and terminal constraints:

$$
\begin{aligned}
P_t^{\mathrm{S}} = P_{\max}\Big[&\frac{1}{2}\left(1+\tanh\big(k(u_t-\delta)\big)\right)u_t \\
&\cdot\frac{1}{2}\left(1+\tanh\big(k(0.995-\mathrm{SOC}_t)\big)\right) \\
&+\frac{1}{2}\left(1-\tanh\big(k(u_t+\delta)\big)\right)u_t \\
&\cdot\frac{1}{2}\left(1+\tanh\big(k(\mathrm{SOC}_t-0.005)\big)\right)\Big], \quad \forall t, \qquad \text{..... (4)}
\end{aligned}
$$

$$
\begin{aligned}
P_{\mathrm{actual},t} = \frac{P_t^{\mathrm{S}}}{2}\Bigg[&\left(1+\tanh\left(\frac{P_t^{\mathrm{S}}}{\epsilon}\right)\right)\eta \\
&+\left(1-\tanh\left(\frac{P_t^{\mathrm{S}}}{\epsilon}\right)\right)\frac{1}{\eta}\Bigg], \quad \forall t, \qquad \text{..... (5)}
\end{aligned}
$$

$$
\mathrm{SOC}_N \geq \mathrm{SOC}_{\mathrm{term}}, \qquad \mathrm{SOC}_{\mathrm{term}} = 0.5. \qquad \text{..... (6)}
$$

Here, u_t is the normalized control command and $P_{\max}$ is the maximum charging and discharging power. The parameter $k = 700$ controls the steepness of the smooth action and SOC gates: increasing k makes their transitions more abrupt, whereas decreasing it produces a wider and more gradual transition. The parameter δ shifts the charging and discharging gates away from zero and thereby defines a smooth deadband for small control commands. The implemented value $\delta = 0.0025$ corresponds to 0.25% of the maximum normalized action magnitude, or a nominal power command of 0.05 kW for $P_{\max} = 20\,\mathrm{kW}$. This small numerical deadband attenuates near-zero commands and reduces switching around zero while leaving the usable action range effectively unchanged.

The parameter η describes the charging and discharging efficiency. The power scale $\epsilon = 1\,\mathrm{kW}$ determines the width of the smooth transition between the charging and discharging efficiency relations in Eq. (5). A larger ϵ produces a more gradual transition around $P_t^{\mathrm{S}} = 0$, while a smaller value more closely approximates an abrupt change between the two operating directions. Equation (4) maps the normalized action and current SOC to grid-side battery power. It progressively restricts charging as SOC approaches 0.995 and discharging as SOC approaches 0.005. Equation (5) maps grid-side power to storage-side power through the smooth efficiency relation.

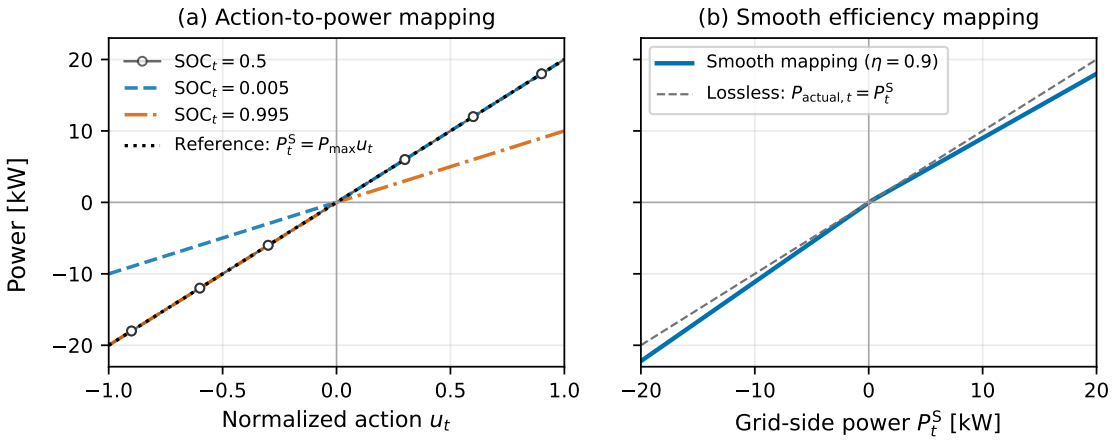


Figure 3: Visualization of the smooth battery mappings in Eqs. (4) and (5). Panel (a) shows grid-side power as a function of the normalized action for $\mathrm{SOC}_t \in \{0.005, 0.5, 0.995\}$. The black dotted line shows the linear reference $P_t^{\mathrm{S}} = P_{\max} u_t$. Panel (b) shows the conversion from grid-side to storage-side power. The plotted parameters are $P_{\max} = 20\,\mathrm{kW}$, $k = 700$, $\delta = 0.0025$, $\eta = 0.9$, and $\epsilon = 1\,\mathrm{kW}$.

As shown in Fig. 3(a), the mapping is approximately $P_t^{\mathrm{S}} = P_{\max} u_t$ when the SOC is far from its limits and the action lies outside the narrow deadband. At the gate locations $\mathrm{SOC}_t = 0.005$ and $\mathrm{SOC}_t = 0.995$, discharging and charging power, respectively, are reduced by one half. Figure 3(b) illustrates the efficiency relation: away from the transition around zero, positive charging power is multiplied approximately by η, whereas negative discharging power is multiplied approximately by $1/\eta$. The hyperbolic tangent provides a differentiable transition between these two regimes.

The battery model has a one-dimensional action space at each time step because the scalar command u_t is the only independently selected control variable. Over the complete horizon, the optimization baseline therefore determines one charging or discharging command for each time step, while battery power and SOC follow from the model equations.

The battery objective is to maximize profit. Let $P_t^+ = \max(P_t^{\mathrm{S}}, 0)$ denote purchased power and $P_t^- = \max(-P_t^{\mathrm{S}}, 0)$ denote sold power. Minimizing the net grid-side cost is equivalent to maximizing profit, giving the general objective

$$\min_{\{u_t\}_{t=0}^{N-1}} J_{\mathrm{grid}} = \frac{1}{1000} \sum_{t=0}^{N-1} \left(S_t^{\mathrm{buy}} P_t^+ - S_t^{\mathrm{sell}} P_t^- \right) \Delta t_{\mathrm{h}}.$$

With $S_t^{\mathrm{buy}} = S_t$ and $S_t^{\mathrm{sell}} = S_t - \Delta_s$, the spread reduces the revenue received for exported electricity. The numerical choice $\Delta_s = 0$ yields $S_t^{\mathrm{buy}} = S_t^{\mathrm{sell}} = S_t$ and recovers $J_{\mathrm{grid}} = \sum_t S_t P_t^{\mathrm{S}} \Delta t_{\mathrm{h}} / 1000$. The factor 1000 converts kWh to MWh so that J_{grid} is expressed in euros. This equation represents the physical economic objective used to compare the methods. In the optimization baseline formulation, the terminal SOC is imposed as the hard constraint in Eq. (6). In the RL formulation, the negative per-step economic contribution is supplemented by a terminal-SOC penalty and, in an additional experimental variant, a progressive terminal-reachability penalty.

These penalties guide policy training but are not included in J_{grid} because they do not represent physical operating costs; their definitions are given in Section 3.2. The battery purchases electricity from the grid when prices are low and sells it back to the grid when prices are high. The main challenges include respecting SOC and power limits, accounting for efficiency losses, handling uncertain future prices, and managing the fact that actions now affect future energy availability. These features make the BESS a suitable test case for reinforcement learning, where a policy must balance using energy now against saving it for likely higher-price periods while always obeying feasibility limits.

3.2 Methodology

This subsection develops the reinforcement-learning formulation for the battery and establishes the implementation, training setup, and evaluation procedure used for both case studies. The corresponding cookie-factory state, action mapping, and reward are specified in Section 4.2. At each time step, the environment maps the policy action to physical setpoints, advances the system dynamics, and returns the next state, reward, and termination signals. For the optimization baseline, IPOPT solves the finite-horizon battery problem described in Section 3.1, with the terminal SOC requirement imposed as a hard constraint.

3.2.1 Reinforcement Learning Formulation

For both use cases, the objective is to determine operating setpoints over a discrete time horizon that optimize the relevant economic objective while respecting all constraints. Let $t \in \{0, \dots, N-1\}$ denote the discrete time index, S_t the spot electricity price, P_t^{S} the grid power, and W_t the available wind power. At each time step, the system is described by a state vector x_t, the agent selects a continuous action vector a_t, and the system evolves to the next state x_{t+1} according to the underlying system dynamics. For the battery case, the only external input is the electricity price S_t.

The reinforcement-learning framework used in this thesis follows the same general structure for both use cases. The environments are implemented using Gymnasium [18], while the continuous-control policies are trained using Stable-Baselines3 [19]. Each interaction produces the transition sample

$$(x_t, a_t, r_t(x_t, a_t), x_{t+1}, d_t)\,,$$

where d_t indicates whether the episode has ended. In the present environments, reaching the end of the fixed scheduling horizon is represented by the `terminated` signal. The transitions are stored in a replay buffer and sampled repeatedly during training, enabling the off-policy learning used by SAC and TD3. This implementation follows the general agent–environment interaction illustrated in Figure 1, extended by forecast-augmented states, environment-specific action mappings, and constraint handling.

For the battery system, the state supplied to the agent consists of the current state of charge, a normalized representation of the current spot electricity price S_t,

the normalized position within the scheduling horizon, and normalized values of the full 24-step price sequence $(S_0, \ldots, S_{N-1})$. Normalization applies to the network observation; S_t in the cost equations retains its physical unit of EUR/MWh. As future work, a rolling-horizon implementation could update the forecast at every decision step. Here, the complete forecast is kept fixed to enable a direct comparison with the optimization baseline over the same single scheduling horizon. The continuous scalar action $u_t \in [-1, 1]$ represents the battery charging or discharging command: positive values correspond to charging and negative values to discharging. The battery environment converts u_t into a requested grid-side power using smooth action and SOC gates, which attenuate charging near $\mathrm{SOC}_t = 0.995$ and discharging near $\mathrm{SOC}_t = 0.005$. A separate storage-side power limit, described below, ensures that the next SOC remains within $[0, 1]$.

During training, the policy action is first processed by an environment-specific action map

$$a_t^{\mathrm{phys}} = \mathcal{M}(x_t, a_t),$$

where $\mathcal{M}$ maps the policy output to physical setpoints and applies environment-specific feasibility handling. The constraints enforced directly by this map differ between the two use cases.

For the battery, the bounded action $u_t \in [-1, 1]$ and the smooth power gates restrict the requested grid-side power. The SOC gates attenuate charging and discharging near the corresponding limits, but do not alone guarantee that the next SOC remains feasible. Therefore, after applying the efficiency mapping, the environment limits the requested storage-side power $\widehat{P}_{\mathrm{actual},t}$ to the energy available within one time step:

$$P_{\mathrm{actual},t} = \mathrm{clip}\left(\widehat{P}_{\mathrm{actual},t}, -\frac{\mathrm{SOC}_t E_{\max}}{\Delta t_{\mathrm{h}}}, \frac{(1 - \mathrm{SOC}_t) E_{\max}}{\Delta t_{\mathrm{h}}}\right),$$

where $\mathrm{clip}(z, l, h) = \min(\max(z, l), h)$. These bounds prevent discharging more energy than is stored or charging beyond the remaining capacity, so the SOC update gives $0 \leq \mathrm{SOC}_{t+1} \leq 1$. If this limit changes the requested power, the corresponding grid-side power is recomputed through the inverse efficiency mapping before calculating the electricity cost. The terminal SOC requirement is handled by reward penalties.

The reward has the same general form in both environments:

$$r_t(x_t, a_t) = -C_t - \sum_{\ell} \lambda_\ell v_{\ell,t} - \lambda_{\mathrm{T}} v_{\mathrm{T},t},$$

where C_t is the grid-electricity cost at time step t, $v_{\ell,t}$ are constraint violations, and λ_ℓ are their corresponding penalty weights. The terminal violation $v_{\mathrm{T},t}$ includes the final-step indicator and is zero before the end of an episode, and λ_{T} is its penalty weight. In the battery case, it represents an insufficient terminal SOC, while in the cookie factory it represents deviation from the cyclic terminal TES temperature. This reward design converts the case-specific economic objective into a return-maximization problem without removing the need for physical feasibility.

More specifically, the battery reward with only a terminal-SOC penalty is

$$r_t(x_t, a_t) = -\frac{S_t P_t^{\mathrm{S}} \Delta t_{\mathrm{h}}}{1000} - \mathbb{1}_{\{t=N-1\}} \lambda_{\mathrm{SOC}} \max(\mathrm{SOC}_{\mathrm{term}} - \mathrm{SOC}_N, 0),$$

where S_t is the spot electricity price in EUR/MWh, P_t^{S} is the grid-side battery power in kW, and Δt_{h} is the step duration in hours. This implemented expression is the $\Delta_s = 0$ special case of the general buy–sell formulation above. Division by 1000 converts kWh to MWh so that the energy charge is expressed in euros. The terminal term is applied only at the end of an episode and penalizes a final SOC below the prescribed minimum. Intermediate SOC feasibility is enforced by the storage-side power limit described above and the subsequent SOC update.

The correspondence with the general reward is explicit:

$$C_t = \frac{S_t P_t^{\mathrm{S}} \Delta t_{\mathrm{h}}}{1000}, \qquad \sum_{\ell} \lambda_\ell v_{\ell,t} = 0,$$

$$\lambda_{\mathrm{T}} = \lambda_{\mathrm{SOC}}, \qquad v_{\mathrm{T},t} = \mathbb{1}_{\{t=N-1\}} \max(\mathrm{SOC}_{\mathrm{term}} - \mathrm{SOC}_N, 0)\,.$$

Thus, the economic term and the terminal penalty have the same structure in both expressions. The running violation sum is zero for this battery variant because intermediate SOC feasibility is enforced directly. The indicator is included in $v_{\mathrm{T},t}$, making the terminal term zero before the final decision step.

As an additional feasibility measure, a progressive terminal-SOC reachability penalty was investigated. At each time step, the environment calculates the minimum SOC required to retain a feasible path to the terminal-SOC target under the maximum possible future charging rate:

$$\mathrm{SOC}_t^{\mathrm{reach}} = \max\left(\mathrm{SOC}_{\mathrm{term}} - (N-t)\frac{\eta P_{\max} \Delta t_{\mathrm{h}}}{E_{\max}}, 0\right).$$

Whenever the current SOC falls below this reachability threshold, the shaped reward is

$$r_t^{\mathrm{reach}}(x_t, a_t) = r_t(x_t, a_t) - \lambda_{\mathrm{reach}} \max\big(\mathrm{SOC}_t^{\mathrm{reach}} - \mathrm{SOC}_t, 0\big)\,,$$

where $\lambda_{\mathrm{reach}} > 0$ weights the reachability penalty. In the general reward, this additional term corresponds to the violation $v_{\mathrm{reach},t} = \max(\mathrm{SOC}_t^{\mathrm{reach}} - \mathrm{SOC}_t, 0)$ with weight λ_{reach}. Unlike the terminal penalty described above, this penalty provides feedback before the final time step and encourages the policy to preserve sufficient energy for the terminal requirement. The resulting feasibility–cost trade-off is reported in Section 3.3.

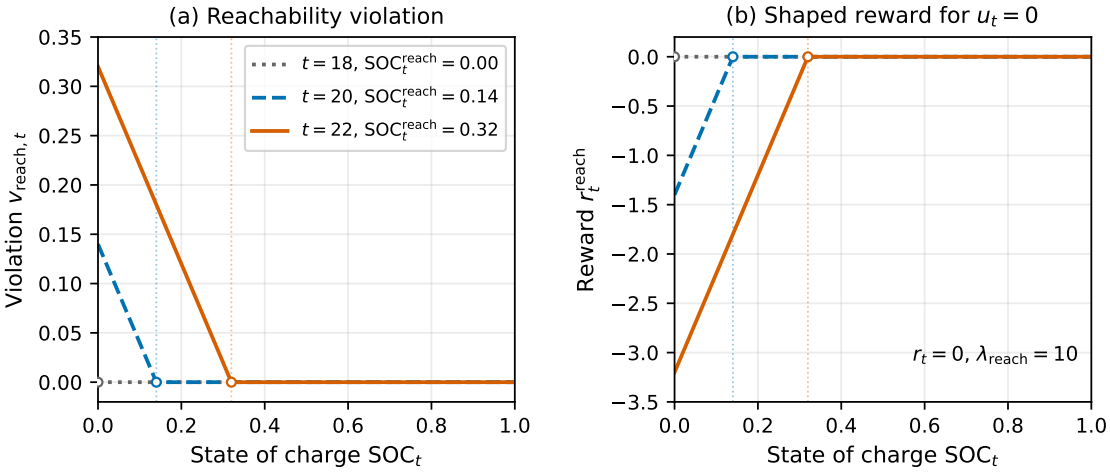


Figure 4: Illustration of the battery reachability violation and shaped reward as functions of SOC at $t \in \{18, 20, 22\}$. The parameters are $N = 24$, $P_{\max} = 20\,\text{kW}$, $E_{\max} = 200\,\text{kWh}$, $\eta = 0.9$, $\Delta t_{\text{h}} = 1\,\text{h}$, $\text{SOC}_{\text{term}} = 0.5$, and $\lambda_{\text{reach}} = 10$. Panel (a) shows $v_{\text{reach},t}$. Panel (b) uses the idle action $u_t = 0$ at the shown nonterminal steps, for which the baseline reward is $r_t = 0$ and hence $r_t^{\text{reach}} = -\lambda_{\text{reach}} v_{\text{reach},t}$. Hollow circles mark the reachability thresholds.

Figure 4 shows that the violation is zero at or above the reachability threshold and increases linearly as SOC falls below it. The shaped reward correspondingly decreases with slope magnitude λ_{reach}. As fewer charging steps remain, the threshold rises and the penalty affects a wider range of SOC values. For a nonzero baseline reward, the reachability term is the same downward adjustment to r_t.

3.2.2 Training Setup

All computations were performed in Python 3.14.1, with Gymnasium 1.2.2 used to implement the reinforcement-learning environments and Stable-Baselines3 2.7.0, based on PyTorch 2.9.1, used to train the RL policies. Experiments were conducted on an AMD Ryzen 7 7840HS CPU with Radeon 780M Graphics.

Training is performed in reinforcement-learning environments before the learned policies are evaluated on fixed test trajectories. The environment provides the transition dynamics, cost calculation, constraint handling, and terminal condition for each episode. The learning algorithm only receives observations, rewards, and termination signals through the environment interface.

For both use cases, one episode corresponds to a 24-step scheduling horizon. During training, the start index of the episode is sampled randomly from the available time-series data. This random-window strategy exposes the agent to different price and renewable-generation patterns instead of repeatedly starting each episode at the same time of day. Both environments use the grid-price series; the cookie factory environment additionally samples the corresponding wind-availability values for the same 24-hour window.

The training environments are wrapped with monitoring and vectorized environment interfaces. Monitoring records episode rewards and lengths for later inspec-

Parameter	Setting for both case studies
Algorithm	SAC and TD3
Policy	MLP policy
Network architecture	SAC: 3×256 ReLU; TD3: 400, 300 ReLU
Replay buffer size	200000
Batch size	256
Learning rate	Linear schedule from $3 \cdot 10^{-4}$
Discount factor γ_{RL}	1
Target-network soft-update coefficient τ	0.005
Episode length	24 steps

Table 1: Final SAC and TD3 training configuration used for both the battery and cookie-factory environments. TD3 uses the Stable-Baselines3 default network architecture.

tion, while the vectorized wrapper provides the interface expected by the Stable-Baselines3 implementation of SAC and TD3. Only one environment instance is used at a time in the current setup, so the vectorization is mainly an interface layer rather than a parallel simulation strategy.

The environment-specific action mappings and reward definitions are given in Sections 3.2.1 and 4.2.1, including the cookie-factory reward in Eq. (15). The cookie-factory penalties are particularly important because its coupled nonlinear feasibility requirements cannot all be represented by simple action bounds.

Hyperparameter selection included 50 tuning trials. The preliminary battery learning-rate study used to select the linear schedule is reported in Section 3.3. The final training settings, which are shared by the battery and cookie-factory case studies, are summarized in Table 1. The environments themselves still differ in their state variables, action spaces, dynamics, and constraint structures.

Both SAC policies use three hidden layers of 256 neurons with ReLU activation functions for both actor and critic networks. TD3 uses the Stable-Baselines3 default architecture of two ReLU hidden layers with 400 and 300 neurons for both actor and critic networks. In all cases, trained models are saved after learning and later loaded for evaluation.

3.2.3 Evaluation Procedure

The trained RL policies are evaluated through deterministic rollouts on fixed 24-hour evaluation windows. For SAC, the mean action of the stochastic policy is selected at each time step. TD3 directly outputs a deterministic action. This produces one reproducible setpoint trajectory for a given price and wind profile and allows direct comparison with the optimization baseline in terms of cost, constraint satisfaction, and computation time.

The primary economic metric is the total net grid-side cost over the evaluation

horizon:

$$J_{\text{grid}} = \sum_{t=0}^{N-1} C_t,$$

where C_t is the grid-electricity cost at time step t. For the battery system, this cost may generally include the fixed buy–sell spread Δ_s defined in Section 3.1; the numerical experiments use $\Delta_s = 0$. Minimizing J_{grid} is equivalent to maximizing profit. For the cookie factory, the cost is computed from the grid power required by the heat-pump system after accounting for available wind power and is minimized directly. Thus, a lower value of J_{grid} indicates higher profit for the battery and lower operating cost for the cookie factory.

The cumulative reward is reported as a secondary metric:

$$R_{\text{ep}} = \sum_{t=0}^{N-1} r_t(x_t, a_t) = -J_{\text{grid}} - \sum_{t=0}^{N-1} \left(\sum_{\ell} \lambda_\ell v_{\ell,t} + \lambda_{\text{T}} v_{\text{T},t} \right).$$

When all penalty terms are zero, the cumulative episode reward equals the negative total grid-electricity cost, $R_{\text{ep}} = -J_{\text{grid}}$. Otherwise, it also reflects penalties for constraint violations.

Because $\gamma_{\text{RL}} = 1$ and the terminal penalty is included in the reward returned with the final transition, this cumulative episode reward contains the same economic and penalty contributions as the realized return G_0. This value is useful for analyzing the RL objective because it combines the economic term with penalty terms. However, it is not used as the only comparison criterion, since the reward may include artificial penalty weights that are not part of the physical economic objective. Therefore, the economic objective and constraint satisfaction are evaluated separately.

Constraint satisfaction is measured separately for each constraint category. For a nonnegative residual magnitude or violation $v_{\ell,t}$ in category ℓ, the total violation over one rollout is defined as

$$V_{\text{tot},\ell} = \sum_{t=0}^{N-1} v_{\ell,t}.$$

Each $V_{\text{tot},\ell}$ retains the unit of its category; violations with different units are not added together. In addition, the fraction of time steps with at least one violation is considered:

$$f_{\text{viol}} = \frac{1}{N} \sum_{t=0}^{N-1} \mathbb{1}_{\{\exists\, \ell:\, v_{\ell,t} > \varepsilon_{\text{viol},\ell}\}},$$

where $\varepsilon_{\text{viol},\ell}$ is a small numerical tolerance expressed in the unit of category ℓ. For the battery system, the relevant checks include SOC bounds, power limits, and the terminal SOC requirement. For the cookie factory, the violation categories include bypass feasibility, complementarity of charging and discharging, TES temperature bounds, grid-power bounds, coupling residuals, and the terminal cyclic TES condition.

The evaluation also records trajectory-level operating indicators. For the BESS, these include the SOC trajectory, commanded and effective battery power, and terminal SOC.

Finally, computation time is used to assess practical applicability. For the optimization baseline, the relevant value is the solver time required to compute a complete schedule for the evaluation horizon. For RL, the one-time training effort is reported separately from the online evaluation time. This distinction is important because RL can require substantial training effort, but a trained policy can usually produce setpoints much faster than solving a nonlinear optimization problem from scratch for every horizon.

Offline training time is excluded from all online computation time comparisons.

3.3 Numerical Results

A preliminary learning-rate study was conducted for SAC using the battery environment. A constant learning rate, a linear-decay schedule, and a power-law-decay schedule were each trained and evaluated over 32 independent runs using the same evaluation setup. The linear schedule achieved the highest mean cumulative reward, 8.9962 ± 0.0286, and the lowest mean objective value, -9.00 ± 0.03 EUR. The corresponding mean rewards were 8.9891 ± 0.0325 for the constant schedule and 8.9809 ± 0.0439 for the power-law schedule.

The differences between the schedules were small relative to the variation across runs, indicating that the battery environment was not highly sensitive to the exact learning-rate decay shape in this experiment. The linear schedule was therefore selected for the subsequent SAC and TD3 training in both case studies and for the comparisons with the optimization baseline.

3.3.1 Comparison with the Optimization Baseline

The trained SAC and TD3 policies were compared with the optimization baseline over 1000 randomly sampled 24-hour price windows. Table 2 summarizes the resulting objective values, computation times, and SOC ranges.

Metric	**SAC policy**	**TD3 policy**	**Optimization baseline**
Mean objective [EUR]	-2.89 ± 1.24	-3.02 ± 1.22	-3.19 ± 1.22
Objective range [EUR]	$[-7.73, -0.35]$	$[-7.78, -0.26]$	$[-7.85, -0.44]$
Mean computation time [s]	0.0126 ± 0.0105	0.0082 ± 0.0067	0.5605 ± 0.6295
Time range [s]	$[0.0102, 0.3444]$	$[0.0070, 0.2150]$	$[0.1164, 9.2371]$
Active offline training time [min]	36	23	Not applicable
Mean maximum SOC	0.8661 ± 0.1620	0.8905 ± 0.1458	0.8793 ± 0.1595
Mean minimum SOC	0.2129 ± 0.1887	0.1947 ± 0.1776	0.1925 ± 0.1796

Table 2: Comparison between the trained SAC and TD3 policies and the optimization baseline over 1000 random 24-hour windows. Lower (more negative) objective values correspond to higher profit.

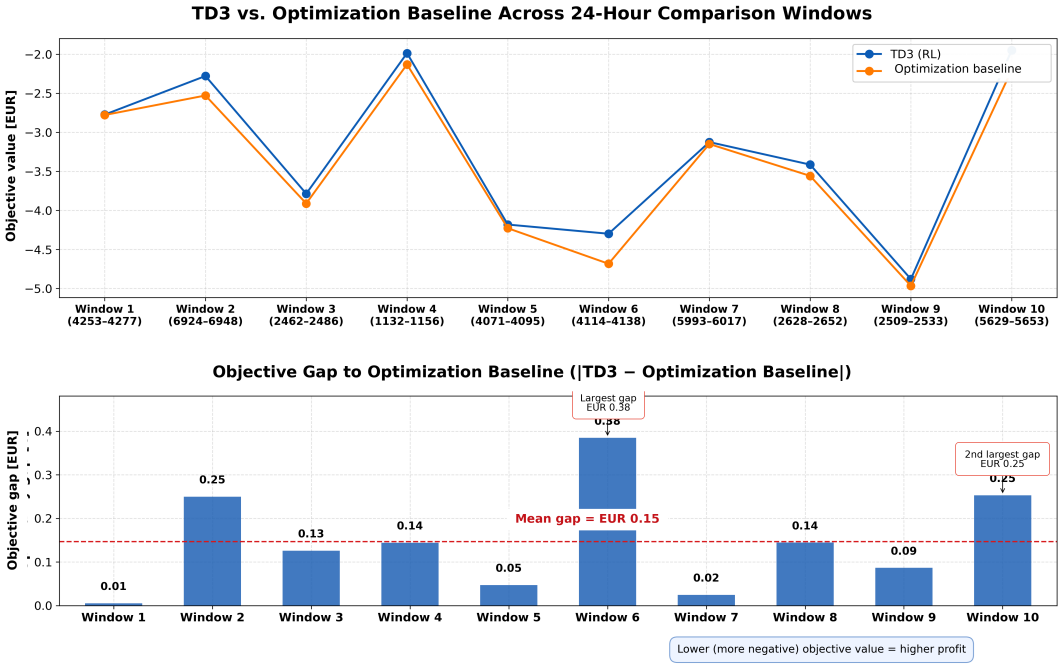


Figure 5: Objective values from TD3 and the optimization baseline for ten randomly selected 24-hour comparison windows. The lower panel shows the corresponding gaps between TD3 and the optimization baseline.

The optimization baseline achieved the lowest mean objective value and therefore the highest mean profit. It was the best-performing method overall. Among SAC and TD3, TD3 performed better. It achieved a lower mean objective value and higher mean profit, and was closer to the optimization baseline. Based on the result from the optimization baseline reported in Table 2, the mean objective gap was 0.16 EUR for TD3 and 0.29 EUR for SAC. Since the objective is minimized, both positive gaps indicate that the RL policies achieved lower profit than the optimization baseline. TD3 performed better in this relatively simple and deterministic environment. This could be explained by TD3 directly maximizing the return, whereas SAC balances return maximization with maintaining policy entropy. Further TD3 results are presented below.

The small objective gaps shown in Figure 5 indicate that the TD3 policy closely approaches the solution from the optimization baseline across the selected comparison windows.

The SOC trajectories are also similar in this comparison. The TD3 policy reached a mean minimum SOC of 0.1947 and a mean maximum SOC of 0.8905, while the solution from the optimization baseline reached a mean minimum SOC of 0.1925 and a mean maximum SOC of 0.8793. This indicates that both methods used a comparable portion of the available storage flexibility.

Figure 6 shows a 24-hour dispatch trajectory for the best TD3 policy and the solution from the optimization baseline. The upper panel compares battery-power profiles and electricity price, while the lower panel compares the corresponding SOC trajectories. Figure 7 separately illustrates the fixed-threshold strategy. This controller charges when the electricity price is at most 15.00 EUR/MWh and discharges

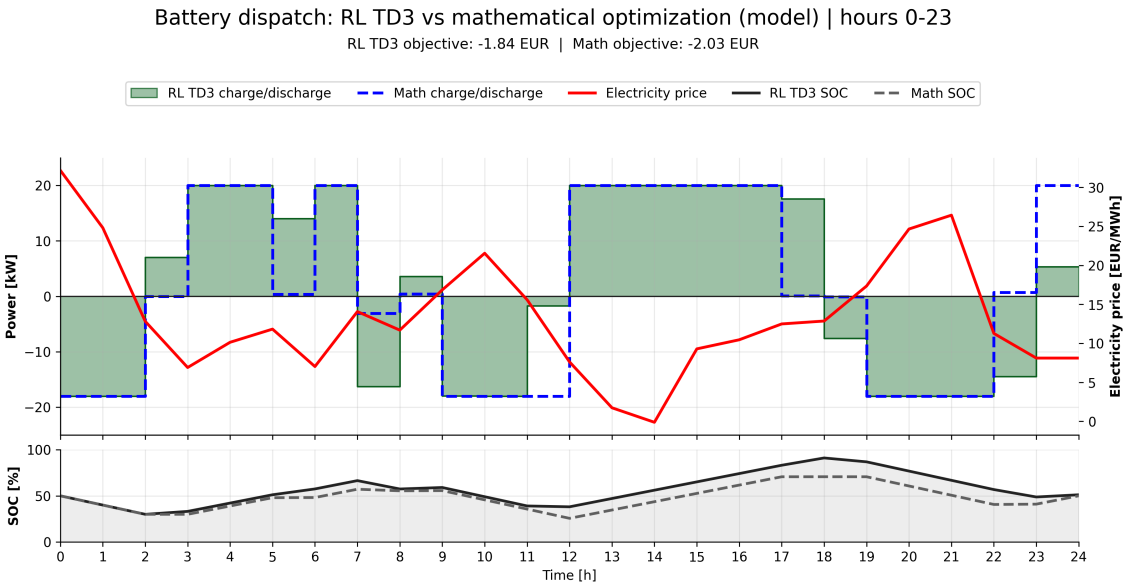


Figure 6: Illustrative 24-hour battery dispatch comparison between the best TD3 policy and the optimization baseline.

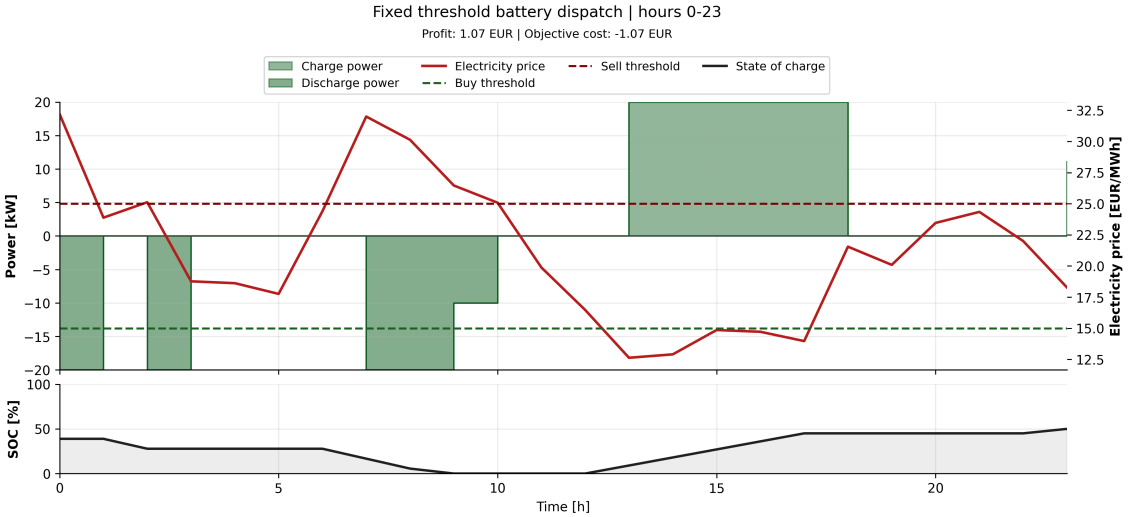


Figure 7: Illustrative 24-hour fixed-threshold battery dispatch. The strategy charges at prices of at most 15.00 EUR/MWh and discharges at prices of at least 25.00 EUR/MWh.

when it is at least 25.00 EUR/MWh.

The fixed-threshold strategy achieved an objective value of −1.07 EUR, corresponding to a profit of 1.07 EUR, and returned the SOC to 0.50 at the end of the illustrated horizon. The TD3 policy achieved a profit of 1.84 EUR in the same window. Based on these displayed values, TD3 produced approximately 72.0% more profit than the fixed-threshold strategy.

In a separate feasibility evaluation, the TD3 policy without the progressive reachability penalty violated the terminal-SOC constraint in 4% of the evaluation windows. Adding the progressive penalty reduced the violation rate to zero but

increased the mean objective gap between TD3 and the optimization baseline to 0.24 ± 0.15 EUR, illustrating the trade-off between strict terminal-constraint satisfaction and economic performance. Upon further investigation, the mean terminal-SOC shortfall without the progressive penalty was only 0.000252. Given its higher profit, this small shortfall was considered tolerable, and the TD3 policy without the progressive reachability penalty was selected as the final model.

The main advantage of the TD3 policy was computation time. The trained TD3 model required, on average, 0.0082 s to generate a 24-hour rollout, while the optimization baseline required 0.5605 s on average to solve the corresponding optimization problem. This corresponds to an average speedup of approximately 68.13×. TD3 required 23 minutes of offline training, which is not included in the online computation time. These results therefore illustrate the central trade-off observed in the battery case: RL provides very fast online setpoint generation after training and closely approaches the result from the optimization baseline, while direct optimization still provides the best objective value.

4 Cookie Production Factory

The second case study considers the cookie production factory as the more complex benchmark, combining multiple controllable components, nonlinear process behavior, thermal energy storage, and coupled operational constraints. As in the battery case, the problem formulation is followed by the methodology and numerical results.

4.1 Problem Formulation

The second use case represents a nonlinear cookie production factory with an electrified steam-generation utility system. The production process requires a constant supply of process heat in the form of superheated steam, while the utility system must decide how to operate the energy conversion components under fluctuating wind generation W_t and electricity prices S_t. The operational objective is to satisfy the steam demand while minimizing grid electricity cost over a finite scheduling horizon. The system model is based on the industrial power-to-heat system introduced by Walden et al. [3] and later used as a renewable steam-generation benchmark by Kyriakidis et al. [9].

The system consists of four main units: a wind turbine (WT), a closed reverse Brayton-cycle high-temperature heat pump (HTHP), a sensible thermal energy storage (TES), and a steam generator (SG). The WT supplies renewable electricity, while additional electricity can be purchased from the grid. The HTHP uses a waste-heat air stream as its heat source and delivers high-temperature process heat to an intermediate heat-transfer-fluid (HTF) loop. The HTF can be routed through the TES or directly to the SG by controllable bypasses. The SG provides the required steam conditions for the consumer factory. The HTHP and SG are represented by polynomial surrogate models, while the TES is modeled as a simplified single energy reservoir, and the charging/discharging behavior is represented using efficiency equations. Figure 8 provides an overview of the complete system, while Fig. 9 shows the detailed HTF flow paths and bypass variables.

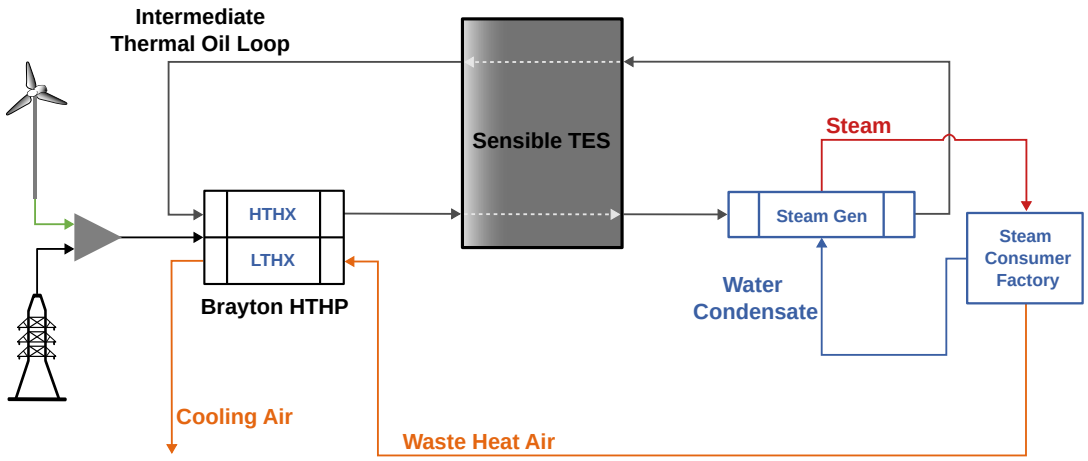


Figure 8: Schematic overview of the electrified steam-generation system serving the cookie production factory. Adapted from Fig. 2 of Kyriakidis et al. [9]; the system was originally introduced by Walden et al. [3].

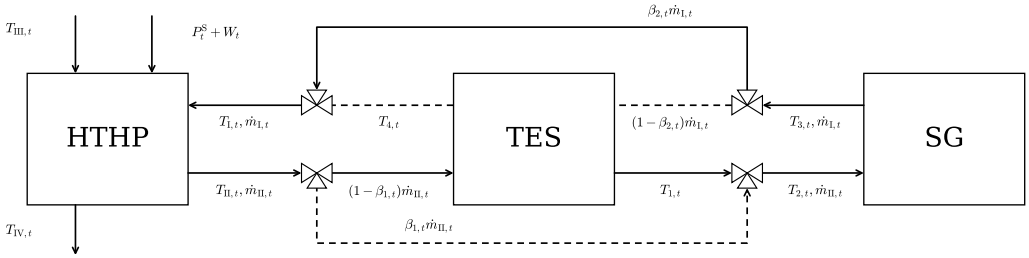


Figure 9: Detailed flow chart of the HTHP, TES, SG, and controllable bypasses in the electrified steam-generation system. Adapted from Fig. 3 of Kyriakidis et al. [9] using the notation of this thesis; the system was originally introduced by Walden et al. [3].

The corresponding finite-horizon optimization problem is formulated over a dis-

crete time horizon with time index $t \in \{0, \ldots, N-1\}$:

$$\min_{\{\beta_{1,t},\beta_{2,t},R_t,\dot{m}_{\mathrm{I},t},T_{\mathrm{I},t}\}_{t=0}^{N-1}} \quad J_{\mathrm{grid}} = \frac{1}{1000}\sum_{t=0}^{N-1} P_t^{\mathrm{S}} S_t \Delta t_{\mathrm{h}}, \qquad \text{..... (7)}$$

$$\text{s.t.} \quad P_t^{\mathrm{S}} + W_t = 3F_{\mathrm{HTHP}}\left(T_{\mathrm{I},t}, \dot{m}_{\mathrm{I},t}, T_{\mathrm{III},t}, R_t\right), \qquad \text{..... (8)}$$

$$T_{\mathrm{II},t} = F_{\mathrm{HTHX}}\left(T_{\mathrm{I},t}, \dot{m}_{\mathrm{I},t}, T_{\mathrm{III},t}, R_t\right),$$

$$T_{\mathrm{IV},t} = F_{\mathrm{LTHX}}\left(T_{\mathrm{I},t}, \dot{m}_{\mathrm{I},t}, T_{\mathrm{III},t}, R_t\right),$$

$$T_{2,t} = \beta_{1,t} T_{\mathrm{II},t} + \left(1-\beta_{1,t}\right) T_{1,t}, \qquad \text{..... (9)}$$

$$T_{\mathrm{I},t} = \beta_{2,t} T_{3,t} + \left(1-\beta_{2,t}\right) T_{4,t}, \qquad \text{..... (10)}$$

$$T_{2,t} = 201.92 + \frac{1819.32}{3\dot{m}_{\mathrm{II},t}},$$

$$T_{3,t} = 196.3 - \frac{188.4}{3\dot{m}_{\mathrm{I},t}},$$

$$\dot{Q}_{s,\mathrm{ch},t} = 3\dot{m}_{\mathrm{II},t} c_{p,\mathrm{f}} \left(T_{\mathrm{II},t} - T_{1,t}\right)\left(1-\beta_{1,t}\right),$$

$$\dot{Q}_{s,\mathrm{dch},t} = 3\dot{m}_{\mathrm{I},t} c_{p,\mathrm{f}} \left(T_{4,t} - T_{3,t}\right)\left(1-\beta_{2,t}\right),$$

$$\dot{Q}_{s,\mathrm{ch},t}\dot{Q}_{s,\mathrm{dch},t} \le \gamma,$$

$$T_{1,t} = T_{\mathrm{II},t} - \epsilon_{\mathrm{ch}}\left(T_{\mathrm{II},t} - T_{s,t}\right),$$

$$T_{4,t} = T_{3,t} - \epsilon_{\mathrm{dch}}\left(T_{3,t} - T_{s,t}\right),$$

$$T_{s,t+1} = T_{s,t} + \frac{\dot{Q}_{s,\mathrm{ch},t} - \dot{Q}_{s,\mathrm{dch},t}}{m_s c_{p,\mathrm{s}}}\Delta t_{\mathrm{s}},$$

$$T_{s,0} = \tilde{T}_0, \qquad T_{s,N} = T_{s,0},$$

$$T_{\mathrm{I},t} \in [177, 250], \qquad \dot{m}_{\mathrm{I},t} \in [5, 16],$$

$$T_{\mathrm{III},t} \in [60, 100], \qquad R_t \in [0.8, 1.53], \qquad \beta_{1,t}, \beta_{2,t} \in [0, 1]. \qquad \text{..... (11)}$$

The label F_4 denotes the charging-side coupling relation in Eq. (9), and F_5 denotes the discharging-side mixing relation in Eq. (10). These are fixed relation labels; the subscript t on each physical variable denotes time. In this formulation, the final subscript t denotes the discrete time step, where $t \in \{0, \ldots, N-1\}$ and N is the number of steps in the scheduling horizon. The objective J_{grid} is the total grid-electricity cost, S_t is the spot electricity price, P_t^{S} is the purchased grid power, and W_t is the available wind power. Thus, Eq. (7) sums the cost $P_t^{\mathrm{S}} S_t \Delta t_{\mathrm{h}}/1000$ over the horizon. Here, Δt_{h} is the step duration in hours, while $\Delta t_{\mathrm{s}} = 3600\Delta t_{\mathrm{h}}$ is used in the thermal-storage balance because the heat rates are expressed in kW and the heat capacities in kJ/(kg K). The subscript beneath the minimum lists the five time-dependent quantities selected independently by the optimization baseline: the two bypass settings $\beta_{1,t}$ and $\beta_{2,t}$, the compressor rotational-speed variable R_t, the mass flow $\dot{m}_{\mathrm{I},t}$, and the HTHP inlet temperature $T_{\mathrm{I},t}$.

In compound subscripts such as those in $T_{\mathrm{I},t}$ and $T_{1,t}$, the Roman labels I–IV and Arabic labels 1–4 identify physical locations in the HTF circuit shown in Fig. 9, whereas the final subscript t is the time index. $T_{\mathrm{I},t}$ and $T_{\mathrm{II},t}$ are respectively the inlet and outlet temperatures of the high-temperature heat exchanger (HTHX), whereas $T_{\mathrm{III},t}$ and $T_{\mathrm{IV},t}$ are the inlet and outlet temperatures of the low-temperature heat exchanger (LTHX). The temperatures $T_{1,t}$ and $T_{4,t}$ are the TES outlet temperatures during charging and discharging, respectively, while $T_{2,t}$ and $T_{3,t}$ are the steam-generator inlet and outlet temperatures. The quantities $\dot{m}_{\mathrm{I},t}$ and $\dot{m}_{\mathrm{II},t}$ denote the

HTHX inlet and outlet HTF mass-flow rates. The implemented closed-loop model uses the same per-HTHP flow variable for these two locations, so $\dot{m}_{\mathrm{II},t} = \dot{m}_{\mathrm{I},t}$. The functions F_{HTHP}, F_{HTHX}, and F_{LTHX} are polynomial surrogate models for the electrical demand of one HTHP, the HTHX outlet temperature, and the LTHX outlet temperature, respectively.

The bypass settings $\beta_{1,t}, \beta_{2,t} \in [0, 1]$ determine the TES operating mode. A value of one fully bypasses the associated storage path: charging uses $\beta_{1,t} < 1$ and $\beta_{2,t} = 1$, discharging uses $\beta_{2,t} < 1$ and $\beta_{1,t} = 1$, and $\beta_{1,t} = \beta_{2,t} = 1$ represents idle operation. $\dot{Q}_{s,\mathrm{ch},t}$ and $\dot{Q}_{s,\mathrm{dch},t}$ are the TES charging and discharging heat-flow rates. The parameter $c_{p,\mathrm{f}}$ is the specific heat capacity of the HTF, while ϵ_{ch} and ϵ_{dch} are the charging and discharging effectiveness factors. The small relaxation parameter γ in $\dot{Q}_{s,\mathrm{ch},t}\dot{Q}_{s,\mathrm{dch},t} \leq \gamma$ prevents simultaneous charging and discharging when it is chosen close to zero.

Finally, $T_{s,t}$ is the TES temperature at the beginning of decision step t, and $T_{s,t+1}$ is the updated temperature after applying the decision at that step. The parameters m_s and $c_{p,\mathrm{s}}$ denote the storage-medium mass and specific heat capacity, respectively. The prescribed initial temperature is $\tilde{T}_0$, so $T_{s,0} = \tilde{T}_0$, and $T_{s,N} = T_{s,0}$ imposes cyclic terminal operation. The final interval relations specify the validity and operating bounds of the surrogate model. The power balance in Eq. (8) and the remaining relations through Eq. (11) collectively describe the surrogate components, bypass mixing, TES dynamics, mutually exclusive charging and discharging, cyclic operation, and admissible operating ranges. All symbols and their units are also collected in the nomenclature.

The cookie-factory optimization model has a five-dimensional action space at each time step. Its independently selected operating variables are the two bypass variables $\beta_{1,t}$ and $\beta_{2,t}$, the compressor-speed-related variable R_t, the mass flow rate $\dot{m}_{\mathrm{I},t}$, and the HTHP inlet temperature $T_{\mathrm{I},t}$. The remaining temperatures, power flows, and storage quantities are coupled through the model equations. In the RL formulation, the action space is reduced to two dimensions: the policy selects a signed storage-mode command and the mass flow rate $\dot{m}_{\mathrm{I},t}$. The environment maps the storage-mode command to $\beta_{1,t}$ and $\beta_{2,t}$ and derives R_t from the F_4 coupling equation. This use case is more complex than the battery system because it combines nonlinear surrogate equations, thermal storage dynamics, and hard process constraints. It therefore provides a suitable second benchmark for evaluating whether reinforcement learning can learn feasible and cost-effective setpoint policies for industrial energy utility systems, and for comparing the learned policy against the optimization baseline.

4.2 Methodology

The cookie-factory optimization problem is reformulated as a finite-horizon MDP using the same agent–environment interaction structure introduced in Section 3.2.1. Its observation, reduced action space, constraint handling, and reward are defined below. For the optimization baseline, IPOPT solves the finite-horizon problem in Section 4.1, including the process constraints and cyclic terminal-storage requirement.

4.2.1 Reinforcement Learning Formulation

For the cookie factory, the observation contains the normalized TES temperature, current electricity price and wind availability, the previous operating mode, the previously applied compressor-speed variable and F_4 residual, the position within the episode, the terminal-temperature gap, the previous action, and the complete price and wind forecasts. The policy action is reduced to

$$a_t = (b_t, \dot{m}_{\mathrm{I},t}), \qquad b_t \in [-1, 1], \quad \dot{m}_{\mathrm{I},t} \in [5, 16],$$

where b_t is a signed TES-mode command. The idle band has a half-width of $\delta_\beta = 0.05$. This value was selected as a small numerical tolerance around zero. It suppresses mode switching caused by small variations in the policy output while retaining most of the normalized action range for active charging and discharging. The value was fixed during the experiments and was not separately optimized. The environment maps b_t to the two physical bypass settings according to Eq. (12):

$$(\beta_{1,t}, \beta_{2,t}) = \begin{cases} \left(1, \dfrac{1+b_t}{1-\delta_\beta}\right), & -1 \le b_t < -\delta_\beta \quad \text{(discharging)}, \\ (1, 1), & |b_t| \le \delta_\beta \quad \text{(idle)}, \\ \left(\dfrac{1-b_t}{1-\delta_\beta}, 1\right), & \delta_\beta < b_t \le 1 \quad \text{(charging)}. \end{cases} \qquad \text{..... (12)}$$

Consequently, the inactive path always has a bypass value of one. The mapping therefore satisfies $\beta_{1,t} + \beta_{2,t} \ge 1$ and prevents simultaneous TES charging and discharging without requiring the policy to learn these two conditions from reward penalties.

The policy selects only b_t and $\dot{m}_{\mathrm{I},t}$; the environment calculates R_t from the resulting bypass settings and the current TES temperature $T_{s,t}$. First, using $\dot{m}_{\mathrm{II},t} = \dot{m}_{\mathrm{I},t}$, it evaluates

$$\begin{aligned} T_{2,t} &= 201.92 + \frac{1819.32}{3\dot{m}_{\mathrm{I},t}}, \\ T_{3,t} &= 196.3 - \frac{188.4}{3\dot{m}_{\mathrm{I},t}}, \\ T_{4,t} &= T_{3,t} - \epsilon_{\mathrm{dch}}(T_{3,t} - T_{s,t}), \\ T_{\mathrm{I},t} &= \beta_{2,t} T_{3,t} + (1 - \beta_{2,t}) T_{4,t}. \end{aligned}$$

These steps determine the steam-generator temperatures and the HTHX inlet temperature without requiring R_t.

Next, the charging-side coupling relation F_4 in Eq. (9) requires $T_{2,t} = \beta_{1,t} T_{\mathrm{II},t} + (1 - \beta_{1,t}) T_{1,t}$. Substituting $T_{1,t} = (1 - \epsilon_{\mathrm{ch}}) T_{\mathrm{II},t} + \epsilon_{\mathrm{ch}} T_{s,t}$ and collecting terms gives

$$T_{2,t} = (1 - \epsilon_{\mathrm{ch}} + \epsilon_{\mathrm{ch}}\beta_{1,t}) T_{\mathrm{II},t} + \epsilon_{\mathrm{ch}}(1 - \beta_{1,t}) T_{s,t}.$$

Solving this linear relation for $T_{\mathrm{II},t}$ gives the required outlet temperature in Eq. (13):

$$T_{\mathrm{II,req},t} = \frac{T_{2,t} - \epsilon_{\mathrm{ch}}(1 - \beta_{1,t}) T_{s,t}}{1 - \epsilon_{\mathrm{ch}} + \epsilon_{\mathrm{ch}}\beta_{1,t}}. \qquad \text{..... (13)}$$

With the implemented $\epsilon_{\text{ch}} = 0.9$ and $\beta_{1,t} \in [0, 1]$, the denominator is at least 0.1, so this inversion is well defined. The environment then chooses R_t so that the HTHX surrogate produces this required temperature:

$$F_{\text{HTHX}}(T_{\text{I},t}, \dot{m}_{\text{I},t}, T_{\text{III},t}, R_t) - T_{\text{II,req},t} = 0, \qquad R_t \in [0.8, 1.53]. \qquad \text{..... (14)}$$

For the fixed inputs $T_{\text{I},t}$, $\dot{m}_{\text{I},t}$, and $T_{\text{III},t}$, collecting powers of R in the HTHX surrogate gives

$$F_{\text{HTHX}}(T_{\text{I},t}, \dot{m}_{\text{I},t}, T_{\text{III},t}, R) = \sum_{j=0}^{3} \kappa_{j,t} R^j,$$

where $\kappa_{j,t}$ are the resulting coefficients at time t. Here, R is a trial value at the fixed time step; R_t denotes the value eventually applied. Equation (14) therefore becomes the scalar cubic

$$\kappa_{3,t} R_t^3 + \kappa_{2,t} R_t^2 + \kappa_{1,t} R_t + \kappa_{0,t} - T_{\text{II,req},t} = 0.$$

The environment computes its roots numerically and retains real roots in $[0.8, 1.53]$. When several such roots exist, it selects the root closest to the previously applied value R_{t-1} for $t \geq 1$. At $t = 0$, the interval midpoint $R_{\text{ref}} = 1.165$ is used as the reference. The applied initial value R_0 is still obtained from the cubic equation. Selecting the nearest root minimizes $|R_t - R_{t-1}|$ only among the available bounded real roots. It is a local continuity heuristic and does not impose a bound on this change. If the previously selected branch no longer provides a bounded real root, R_t may change abruptly. Switching between TES charging and discharging modes is determined by b_t and is not prevented by this root-selection rule.

If no real root satisfies the bounds, the environment evaluates the absolute surrogate mismatch at both interval endpoints and at all real stationary points within the interval. The stationary points satisfy

$$3\kappa_{3,t} R^2 + 2\kappa_{2,t} R + \kappa_{1,t} = 0.$$

It applies the candidate with the smallest absolute mismatch, using distance to the reference value to break ties. This fallback preserves the bounds on R_t while leaving a possible coupling violation. After selecting R_t, the environment recomputes $T_{\text{II},t}$ and $T_{1,t}$ and measures the remaining mismatch with the residual of Eq. (9), labeled F_4:

$$v_{F_4,t} = \left| T_{2,t} - \beta_{1,t} T_{\text{II},t} - (1 - \beta_{1,t}) T_{1,t} \right|.$$

For the cookie factory, the environment clips b_t and $\dot{m}_{\text{I},t}$ to their allowed intervals and maps b_t to bypass values in $[0, 1]$. The inactive storage path always has a bypass value of one, which guarantees $\beta_{1,t} + \beta_{2,t} \geq 1$ and prevents simultaneous charging and discharging. The derived compressor-speed variable is also restricted to $R_t \in [0.8, 1.53]$. TES and inlet-temperature bounds, heat-flow and grid-power bounds, any remaining F_4 coupling residual, and the cyclic terminal-temperature requirement are represented by the reward penalties defined below. These penalties guide learning but do not guarantee constraint satisfaction.

The cookie-factory reward contains the economic cost and the constraints that are not guaranteed by the action mapping. Define the positive-part operator by $[\,\cdot\,]_+ = \max(\,\cdot\,, 0)$, $\Delta T_s = T_s^{\max} - T_s^{\min}$, and the violations

$$
\begin{aligned}
v_{s,t} &= [T_s^{\min} - T_{s,t+1}]_+ + [T_{s,t+1} - T_s^{\max}]_+, \\
v_{\mathrm{I},t} &= [177 - T_{\mathrm{I},t}]_+ + [T_{\mathrm{I},t} - 250]_+, \\
v_{Q,t} &= [-\dot{Q}_{s,\mathrm{ch},t}]_+ + [-\dot{Q}_{s,\mathrm{dch},t}]_+ + [\dot{Q}_{s,\mathrm{ch},t} - \dot{Q}_s^{\max}]_+ + [\dot{Q}_{s,\mathrm{dch},t} - \dot{Q}_s^{\max}]_+, \\
v_{P,t} &= [-P_t^{\mathrm{S}}]_+ + [P_t^{\mathrm{S}} - P_{\mathrm{S}}^{\max}]_+.
\end{aligned}
$$

Here, $v_{s,t}$ measures violations of the TES temperature bounds, $v_{\mathrm{I},t}$ the HTHX inlet-temperature bounds, $v_{Q,t}$ the TES charging and discharging heat-flow bounds, and $v_{P,t}$ the grid-power bounds. Each term is zero when its corresponding constraint is satisfied. For the one-hour time steps used here, the grid cost is $C_t = S_t P_t^{\mathrm{S}}/1000$. The implemented reward is

$$
\begin{aligned}
r_t^{\mathrm{cookie}}(x_t, a_t) = \; & - C_t - \frac{\lambda_s}{\Delta T_s} v_{s,t} - \frac{\lambda_{\mathrm{I}}}{250 - 177} v_{\mathrm{I},t} - \frac{\lambda_Q}{\dot{Q}_s^{\max}} v_{Q,t} - \frac{\lambda_{F_4}}{\Delta T_s} v_{F_4,t} \\
& - \frac{\lambda_P}{P_{\mathrm{S}}^{\max}} v_{P,t} - \frac{\lambda_{\mathrm{cyc}}}{\Delta T_s} v_{\mathrm{T},t}. \qquad \text{..... (15)}
\end{aligned}
$$

Here, $v_{\mathrm{T},t} = \mathbb{1}_{\{t=N-1\}} |T_{s,N} - T_{s,0}|$ is the terminal violation. The coefficients multiplying the violations in Eq. (15) are the corresponding penalty weights in the general reward. The values used in training are $T_s^{\min} = 183\,^{\circ}\mathrm{C}$, $T_s^{\max} = 324\,^{\circ}\mathrm{C}$, $\dot{Q}_s^{\max} = 5000\,\mathrm{kW}$, $P_{\mathrm{S}}^{\max} = 5000\,\mathrm{kW}$, $\lambda_s = 20$, $\lambda_{\mathrm{I}} = 10$, $\lambda_Q = 5000$, $\lambda_{F_4} = 500$, $\lambda_P = 3000$, and $\lambda_{\mathrm{cyc}} = 3000$. These coefficients were selected empirically during training. The comparatively large λ_Q and λ_P strongly discourage heat-flow and grid-power violations. The terminal coefficient λ_{cyc} is also large because its penalty is applied only once, at the end of the episode. The smaller λ_s and λ_{I} reflect that the storage-state and inlet-temperature bounds were largely satisfied during preliminary runs. These values are fixed engineering choices rather than proven optimal coefficients. Thus, the physical operating cost remains identifiable within the reward, while the penalties provide training feedback for state, inlet-temperature, heat-flow, F_4-coupling, grid-power, and terminal-cyclic violations.

4.2.2 Training and Evaluation Setup

The cookie-factory policies use the training procedure in Section 3.2.2 and the shared SAC and TD3 settings in Table 1. Each episode spans 24 hourly steps, with matched electricity-price and wind-power windows sampled during training. The linear learning-rate schedule selected in the battery study is also used here.

The policies are evaluated using the procedure and metrics in Section 3.2.3. SAC uses the mean policy action and TD3 uses its deterministic action, with the same electricity-price and wind-power data supplied to the policies and the optimization baseline. The physical grid-electricity objective in Eq. (7) is reported separately from reward penalties. Constraint violations are reported by category in their original units, including the F_4 coupling residual, heat-flow and grid-power bounds, and terminal cyclic-storage deviation. Offline training effort and online schedule-generation time are reported separately.

4.3 Numerical Results

SAC, TD3, and the optimization baseline were evaluated using the same 200 randomly selected 24-hour windows, with identical electricity-price and wind-power data provided to each method. Table 3 summarizes the resulting objective values, computation times, and constraint violations for all 200 comparisons.

Metric	**SAC policy**	**TD3 policy**	**Optimization baseline**
Mean objective [EUR]	2092.58 ± 667.56	2081.35 ± 685.52	2012.24 ± 709.83
Objective range [EUR]	$[-186.86, 3578.93]$	$[-482.44, 3590.87]$	$[-888.04, 3486.53]$
Mean computation time [s]	0.0217 ± 0.0282	0.0180 ± 0.0154	11.5901 ± 4.5103
Time range [s]	$[0.0132, 0.4026]$	$[0.0099, 0.2182]$	$[4.6821, 30.4373]$
Active offline training time [h]	2.81	1.24	Not applicable
Mean F_4 residual [°C]	0.4555	0.5300	0
Mean heat-flow-bound violation [kW]	0.5989	3.5719	0
Mean grid-power violation [kW]	98.3749	68.5787	0
Mean terminal cyclic deviation [°C]	1.8380	2.1386	0

Table 3: Comparison of the trained SAC and TD3 policies with the optimization baseline over 200 matched 24-hour cookie-factory windows. Lower objective, computation time, and constraint values are better. Constraint quantities are means per window and are reported separately because they have different physical units.

SAC achieved a mean cost of 2092.58 ± 667.56 EUR, compared with 2012.24 ± 709.83 EUR for the optimization baseline. Based on the value from the optimization baseline reported in Table 3, its mean cost gap was 80.33 EUR and its mean absolute relative gap was 6.31%. SAC required 0.0217 s per schedule, whereas the optimization baseline required 11.5901 s, corresponding to an online speedup of approximately 533.81×. For SAC, the mean values per window were 0.4555 for the F_4 coupling residual, 0.5989 for heat-flow-bound violations, 98.3749 for grid-power violations, and 1.8380 for the terminal cyclic-storage deviation. The corresponding values from the optimization baseline were all 0.

TD3 achieved a mean cost of 2081.35 ± 685.52 EUR. Based on the value from the optimization baseline reported in Table 3, its mean cost gap was 69.11 EUR and its mean absolute relative gap was 5.66%. TD3 required 0.0180 s per schedule, corresponding to an online speedup of approximately 645.06×. For TD3, the mean values per window were 0.5300 for the F_4 coupling residual, 3.5719 for heat-flow-bound violations, 68.5787 for grid-power violations, and 2.1386 for the terminal cyclic-storage deviation. Inlet-temperature and state-bound violations were zero on average for both RL policies.

TD3 achieved the smaller economic gap of the two RL policies. However, nei-

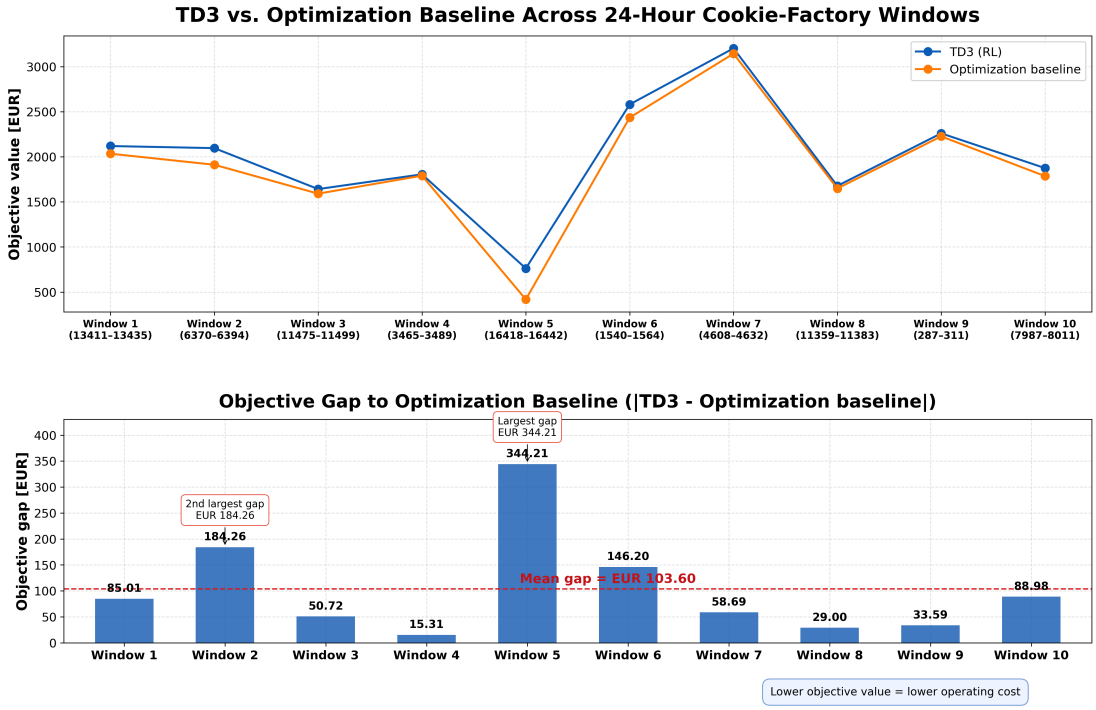


Figure 10: Objective values from TD3 and the optimization baseline for the first ten matched 24-hour cookie-factory windows. The lower panel shows the corresponding absolute objective gaps between TD3 and the optimization baseline.

ther policy matched the feasibility of the optimization baseline. TD3 produced the smaller grid-power violation, whereas SAC produced smaller F_4 residuals, heat-flow-bound violations, and terminal cyclic deviations. Since these constraint quantities have different physical units, they are reported separately rather than combined into a single feasibility score. Despite the remaining violations, both trained policies generated complete schedules substantially faster than the optimization baseline.

Figure 10 shows the objective values from TD3 and the optimization baseline and their absolute differences for ten comparisons. The optimization baseline values are those reported in Table 3. Since the TD3 schedules contain constraint violations, the lower-panel quantities are descriptive objective gaps and should not be interpreted as optimality gaps between equally feasible solutions.

Figure 11 illustrates the schedules produced for one 24-hour window. In this example, TD3 obtained a cost of 1244.27 EUR, while the optimization baseline obtained 1130.70 EUR, corresponding to a gap of 113.57 EUR, or approximately 10.0% relative to the result from the optimization baseline. The upper panel shows the TD3 power balance, including the available wind power, grid power, HTHP power demand, and time-varying electricity price. The lower panel directly compares the grid-power schedules. The optimization baseline concentrates grid consumption more strongly in lower-price periods and reduces it around the highest-price hours, whereas TD3 follows a smoother profile and continues purchasing more grid electricity during some higher-price periods.

These results show that the optimization baseline retains an advantage in both operating cost and feasibility. Nevertheless, both trained policies achieve operating

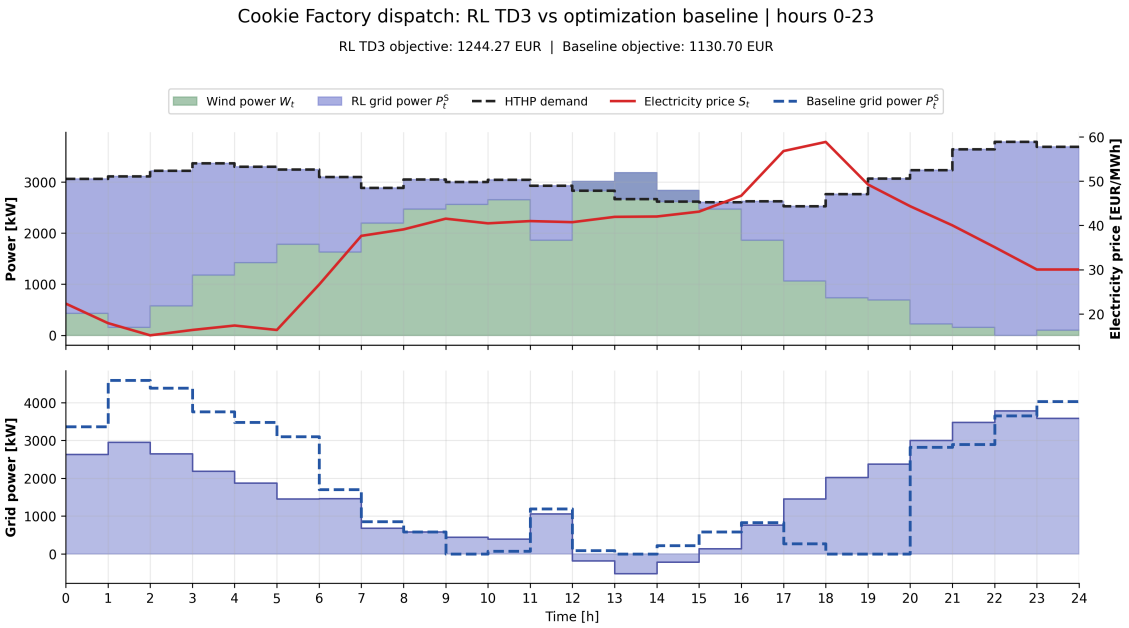


Figure 11: Illustrative 24-hour cookie-factory dispatch comparison between the TD3 policy and the optimization baseline. The upper panel shows wind power W_t, grid power P_t^{S}, total HTHP demand $3F_{\mathrm{HTHP}}$, and electricity price S_t; the lower panel compares the grid-power profiles from TD3 and the optimization baseline.

costs close to the optimization baseline while generating setpoints rapidly.

5 Conclusion and Future Work

5.1 Conclusion

This thesis investigated whether reinforcement learning can determine feasible and cost-effective operating setpoints for energy utility systems of different complexity. The setpoint-selection tasks were formulated as finite-horizon Markov decision processes and evaluated for a battery energy storage system and the electrified steam-generation utility system of a cookie production factory. The battery represents a one-dimensional storage-control problem, whereas the cookie-factory optimization model contains five coupled control variables and nonlinear process constraints. For the cookie-factory RL formulation, the action space was reduced to two dimensions by allowing the policy to select a signed storage-mode command and mass flow while deriving the remaining quantities through the process equations. In both cases, the trained policies were compared with the optimization baseline using the original economic objective, constraint satisfaction, and online computation time.

For the battery system, the optimization baseline achieved the highest mean profit, but the TD3 policy approached its performance closely. Across 1000 randomly sampled 24-hour windows, the mean objective was -3.02 EUR for TD3 and -3.19 EUR for the optimization baseline, giving a mean gap of 0.16 EUR. TD3 also produced SOC trajectories that used a similar portion of the available storage range. Its main advantage was speed: a complete policy rollout required 0.0082 s on average, compared with 0.5605 s for the optimization baseline, corresponding to a speedup of approximately $68.13\times$.

The cookie factory presented a more difficult control problem because nonlinear surrogate equations, storage dynamics, and process constraints must be satisfied simultaneously. Across the 200 matched comparisons with the optimization baseline, SAC produced a mean cost gap of 80.33 EUR and a mean absolute relative gap of 6.31%, while TD3 produced a smaller mean gap of 69.11 EUR and a smaller mean absolute relative gap of 5.66%. SAC and TD3 generated schedules in 0.0217 s and 0.0180 s on average, corresponding to speedups of approximately $533.81\times$ and $645.06\times$, respectively. TD3 is therefore selected as the better cookie-factory RL policy overall based on its economic performance, online speed, and smaller grid-power violation. However, SAC yielded smaller F_4 residuals, heat-flow-bound violations, and terminal cyclic deviations. Both policies remained substantially less feasible than the optimization baseline.

These online speed advantages required offline training. Training the battery policies required 36 minutes for SAC and 23 minutes for TD3, while training the cookie-factory policies required 2.81 hours and 1.24 hours, respectively.

The results therefore provide a qualified answer to the research question. Reinforcement learning can learn continuous setpoint policies that produce economically meaningful schedules with very low online computation time. Its practical value is strongest when decisions must be generated repeatedly under short computation time limits and when offline training can be performed in advance. In the simpler battery system, the learned policy remained close to the optimization baseline. As

system complexity and constraint coupling increased, the gap in operating cost and feasibility became more pronounced. The optimization baseline remained superior when the best objective value and reliable constraint satisfaction were the primary requirements.

5.2 Future Work

Constraint handling should be strengthened for the cookie-factory policy. In particular, the observed grid-power violations should be addressed through a constraint-preserving action mapping or action projection. The two-dimensional action formulation should also be investigated further to determine how the derived compressor-speed calculation and storage-mode mapping affect feasibility, sensitivity, and learning stability.

The present evaluation uses fixed 24-hour price and wind profiles that are available to both methods. A rolling-horizon implementation should instead update forecasts and setpoints as new measurements become available. This would allow the policies to be tested under forecast errors, measurement noise, disturbances, and model–plant mismatch. Domain randomization or targeted retraining could then be used to improve robustness to operating conditions not represented in the original training data.

The comparison with the optimization baseline can also be extended. The schedule generated by the RL policy could be used to warm-start IPOPT, while hybrid methods such as BO-IPOPT should be evaluated on the same windows and computation budgets. Such hybrid approaches could use rapidly generated RL setpoints to initialize a subsequent optimization step that restores feasibility and improves the objective. Finally, the proposed policies should be validated using a higher-fidelity real-plant implementation.

References

[1] M. Khalid. "Smart grids and renewable energy systems: Perspectives and grid integration challenges". In: *Energy Strategy Reviews* 51 (2024), p. 101299. DOI: 10.1016/j.esr.2024.101299.

[2] A. Wächter and L. T. Biegler. "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming". In: *Mathematical Programming* 106 (2006), pp. 25–57. DOI: 10.1007/s10107-004-0559-y.

[3] J. V. Walden et al. "Nonlinear operational optimization of an industrial power-to-heat system with a high temperature heat pump, a thermal energy storage and wind energy". In: *Applied Energy* 344 (2023), p. 121247. DOI: 10.1016/j.apenergy.2023.121247.

[4] M.-H. Lin, J.-F. Tsai, and C.-S. Yu. "A Review of Deterministic Optimization Methods in Engineering and Management". In: *Mathematical Problems in Engineering* 2012 (2012), p. 756023. DOI: 10.1155/2012/756023.

[5] A. Zhigljavsky and A. Žilinskas. *Stochastic Global Optimization*. Vol. 9. Springer Optimization and Its Applications. New York, NY: Springer, 2008. ISBN: 978-0-387-74022-5. DOI: 10.1007/978-0-387-74740-8.

[6] L. Kyriakidis, R. Kansara, and M. I. Roldán Serrano. "Energy management of industrial energy systems via rolling horizon and hybrid optimization: A real-plant application in Germany". In: *Energies* 18.15 (2025), p. 3977. DOI: 10.3390/en18153977.

[7] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018. URL: http://incompleteideas.net/book/the-book-2nd.html.

[8] E. Pilling, M. Bähr, and R. Wunderlich. *Reinforcement Learning Methods for the Stochastic Optimal Control of an Industrial Power-to-Heat System*. Dec. 2025. DOI: 10.48550/arXiv.2411.02211. URL: http://arxiv.org/abs/2411.02211 (visited on 02/11/2026).

[9] L. Kyriakidis, M. A. Mendez, and M. Bähr. "A hybrid algorithm based on Bayesian optimization and Interior Point OPTimizer for optimal operation of energy conversion systems". In: *Energy* 312 (2024). Licensed under CC BY 4.0; Fig. 2 adapted and Fig. 3 reproduced with cropping, p. 133416. DOI: 10.1016/j.energy.2024.133416. URL: https://elib.dlr.de/207751/ (visited on 09/13/2026).

[10] S. Fujimoto, H. van Hoof, and D. Meger. "Addressing function approximation error in actor-critic methods". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 1587–1596. URL: https://proceedings.mlr.press/v80/fujimoto18a.html.

[11] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 1861–1870.

[12] A. P. Tran and F. C. Yücel. "Safe and efficient control of a Brayton cycle heat pump using reinforcement learning". In: *Proceedings of the 16th International Modelica&FMI Conference*. Vol. 218. Linköping University Electronic Press, Oct. 2025, pp. 1017–1026. DOI: 10.3384/ecp2181017. URL: https://ecp.ep.liu.se/index.php/modelica/article/view/1389.

[13] P. H. Takam and N. Fruiba. *Stochastic optimal control problems for the cost-optimal management of a standalone microgrid*. Oct. 2025. DOI: 10.48550/arXiv.2511.00167. URL: https://arxiv.org/abs/2511.00167.

[14] Q. Meng, S. Hussain, F. Luo, Z. Wang, and X. Jin. "An online reinforcement learning-based energy management strategy for microgrids with centralized control". In: *IEEE Transactions on Industry Applications* 61.1 (2025), pp. 1501–1510. DOI: 10.1109/TIA.2024.3430264.

[15] J. Nocedal and S. J. Wright. *Numerical Optimization*. 2nd ed. New York: Springer, 2006. DOI: 10.1007/978-0-387-40065-5.

[16] L. Kyriakidis, M. Bähr, and M. A. Mendez. "Enhanced hybrid algorithm based on Bayesian optimization and Interior Point OPTimizer for constrained optimization". In: *Optimization and Engineering* (2025), pp. 1–52. DOI: 10.1007/s11081-025-09975-y.

[17] N. Bäuerle and U. Rieder. "Theory of Finite Horizon Markov Decision Processes". In: *Markov Decision Processes with Applications to Finance*. Universitext. Berlin, Heidelberg: Springer, 2011, pp. 13–57. DOI: 10.1007/978-3-642-18324-9_2.

[18] M. Towers et al. "Gymnasium: A standard interface for reinforcement learning environments". In: *arXiv preprint arXiv:2407.17032* (2024). DOI: 10.48550/arXiv.2407.17032. URL: https://arxiv.org/abs/2407.17032.

[19] A. Raffin et al. "Stable-Baselines3: Reliable reinforcement learning implementations". In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: https://www.jmlr.org/papers/v22/20-1364.html.